

SCIENCE HUB

The Complete Technical & Business Documentation ("The Science Hub Bible")

GST Billing & Inventory Management System Prepared for engineering, operations, and business stakeholders
Third Edition | Updated 2026-08-17 | Verified directly against the live source code

Table of Contents

1. Executive Summary
2. High-Level System Overview
3. Complete Technology Stack
4. Folder Structure
5. Application Flow
6. User Roles & Permissions
7. Authentication & Sessions
8. Database (Full ER Reference)
9. Complete Module Documentation
10. Invoice Flow — Step by Step
11. Stock Flow & the Movement Ledger
12. GST Logic
13. API Documentation (Full Reference)
14. Frontend Architecture
15. Backend Architecture
16. Data Flow
17. PDF Generation
18. Email System
19. Security
20. Error Handling
21. Deployment
22. Third-Party Services
23. Environment Variables

- 24. Project Dependencies
- 25. Business Workflow (A Day in the Life)
- 26. Non-Technical Explanation
- 27. Developer Guide
- 28. Future Improvements

How this document was built: every claim in this document was verified by reading the actual source code at `d:\nextApps\science-hub` — not guessed, and not copied blindly from the project's own internal notes (`CLAUDE.md` , `DEVELOPMENT_LOG.md`). Places where those internal notes were found to be outdated, and later corrected, are called out explicitly in "Update" callouts throughout, so this Bible reflects the application as it truly runs today, 2026-08-17.

What's new in this Third Edition (since the 2nd Edition, 2026-07-24): a full standalone **Rate Lists** module — a downloadable, shareable price-sheet/catalog builder deliberately independent of the Product catalog, with PDF generation/email/bulk Excel import/export (Ch. 9.18); an admin-configurable **document numbering** system for invoices, purchase bills, and credit notes, with three selectable layouts and Indian-Financial-Year-based resets (Ch. 9.19, 12); a **transportation charge** line on invoices and purchase bills, kept out of the CGST/SGST/IGST split and always server-recomputed (Ch. 9.1, 9.7, 12.4); **GST-safe Bin retention** — invoices, purchase bills, and credit notes are now exempt from the 30-day auto-purge so a GST document-number sequence gap can never silently appear (Ch. 9.13); an editable "Bill To" customer directly from Invoice Edit (Ch. 9.1); the `UnitCombo` free-text-plus-suggestions component replacing constrained unit `Select s` app-wide except inside the Rate List items table (Ch. 9.20); a single app-wide fullscreen `Modal` component now used at 19+ popup call sites, replacing inline-expanding Settings sections (Ch. 9.12, 9.20, 14.2); a mobile-breakpoint and list-pagination UX pass (Ch. 9.20, 14.4); a corrected `bustCache` / `bustCachePrefix` cache-busting distinction after a real stale-cache bug was found and fixed (Ch. 16.1); Rate List **Bin integration** (soft-delete restore/permanent-delete) shipped shortly after the Rate Lists module itself, closing a gap this document's own working notes had briefly called "not yet supported" (Ch. 9.13, 9.18); a `.xlsx` export option for the GST filing package (Ch. 9.11, 13); and a login-flow fix so a transient database outage during a login attempt is now handled gracefully rather than crashing (Ch. 7, 19). Also note: while researching this edition, `resolveNumberFormat()` 's fallback was briefly found to default to `seq_fy` (`18/2026-27`) rather than the `prefix_fy_seq` (`SH-2026-27-0001`) style the project's own internal notes always described as default — a drift introduced mid-development and then reverted the same day this edition was finalized (2026-08-17), once flagged, so that a brand-new business's first invoice/bill/credit-note numbers `SH-2026-27-0001` -style by default, exactly as documented, unless an admin explicitly picks a different layout in Settings → Document Numbering (Ch. 8, 9.19).

1. Executive Summary

What is Science Hub?

Science Hub is a web-based business management system built for a company that sells laboratory and scientific supplies (chemicals, glassware, instruments, safety equipment, consumables). In plain terms, it is the digital replacement for the paper ledgers, Excel sheets, and manual invoice books a supplies business would otherwise use to run itself day to day.

It covers the full commercial cycle of such a business:

- Keeping a catalogue of products, their brands, categories, prices, and how much stock is on the shelf.
- Creating GST-compliant sales invoices for customers, automatically calculating the correct tax.
- Recording payments as customers pay, and knowing at a glance who still owes money.
- Handling product returns from customers by issuing credit notes.
- Buying stock from vendors/suppliers, recording purchase bills, and paying those bills.
- Keeping an audit-proof ledger of every single stock movement — every sale, purchase, return, edit, and cancellation.
- Producing management reports — sales, purchases, outstanding balances, GST summaries, and a ready-to-file GST return package.
- Building and sharing standalone price-sheets/rate lists with customers, independent of the product catalogue.
- Emailing professional PDF invoices, and now rate lists, directly to customers.
- Controlling who on staff can see or touch what, with a full audit trail of every action anyone takes.
- A "recycle bin" so that accidental deletions are never permanently lost — indefinitely for legally-numbered documents (invoices, purchase bills, credit notes), 30 days for everything else.

What business problem does it solve?

A scientific-supplies trading business deals with GST tax rules, stock that must never be oversold, customers who pay in instalments, and a legal requirement to keep clean paperwork for tax filing. Doing this by hand (or in loose spreadsheets) is slow and error-prone: invoice numbers get duplicated, stock counts drift from reality, GST math is done inconsistently, and there is no record of who changed what. Science Hub solves all of this by making every one of these steps a single, auditable, automatically-calculated action inside one system.

Who uses it?

Role	Who they typically are	What they can do
ADMIN	Business owner / senior manager	Everything — full read/write access to all data, user management, business settings, bank details, document-numbering configuration, permanently deleting records, GST filing, and manual stock adjustments.
STAFF	Day-to-day billing/accounts clerk	Full read/write on invoices, customers, products, purchases, payments, rate lists, etc. Can be individually granted or denied access to six specific "sections" (overview dashboards, reports, and payment histories) by an admin.

Role	Who they typically are	What they can do
MANAGER	Supervisor who needs visibility but not editing rights	Same section-based visibility model as staff, but is blocked from creating, editing, or deleting anything anywhere in the system — a strict read-only viewer.

Why was it built?

The codebase and its own internal documentation make clear this is a purpose-built, single-tenant system for one specific business (not a generic multi-company SaaS product) — the system is deployed once, for one organisation, on Vercel with one Neon database.

Main features (at a glance)

- Invoice creation, editing, and soft-deletion, numbered in an **admin-configurable layout** (Settings → Document Numbering) rather than one fixed shape — a business can keep its own pre-existing paper numbering (e.g. `18/2026-27`) or switch to the app's own `SH-2026-27-0001` style.
- Automatic GST computation (CGST+SGST for in-state sales, IGST for inter-state sales), server-verified independently of the browser.
- An optional per-document **transportation/freight charge** line, with its own GST rate and amount, shown separately from the item-level tax on both invoices and purchase bills.
- Product returns issued as numbered credit notes, same configurable-layout numbering as invoices.
- Purchase bills from vendors with the same configurable numbering, attachments, and a cancel/un-cancel workflow.
- A single, append-only stock movement ledger recording every reason stock ever changed.
- **Rate Lists** — a standalone, downloadable price-sheet/catalogue builder for sharing quotations with customers, deliberately independent of the Product catalogue, with bulk import (Excel paste or file upload), PDF generation, email send, and Excel export.
- Section-based permissions so an admin can fine-tune what a given staff/manager account can view.
- Recycle Bin — 30-day grace period and automatic purge for most record types; invoices, purchase bills, and credit notes are retained indefinitely instead, since their numbers are legally significant GST sequences.
- Global search across invoices, customers, products, vendors, purchase bills, brands, and categories.
- A dedicated GST filing package (Sales Register, Purchase Register, Credit Notes, HSN Summary) exportable as a ZIP for handing to an accountant.
- Business branding (name, tagline, logo) that appears on invoices, rate lists, the login page, and the browser tab.
- Full activity log of every meaningful action, by whom and when.
- Email delivery of invoice and rate-list PDFs straight from the app via Gmail.
- A default-deny API gatekeeper (`middleware.ts`) protecting every backend route as a safety net alongside each route's own permission check.

Update: during the research for this edition, `resolveNumberFormat()` 's fallback (`src/lib/documentNumbering.ts`) was briefly found to default to `seq_fy` (`18/2026-27` , no prefix, no zero-padding) rather than the `prefix_fy_seq` style (`SH-2026-27-0001`) the project's own internal documentation has always described as the default — a drift introduced mid-development (see `DEVELOPMENT_LOG.md` Era 9) and never carried into `CLAUDE.md` . On 2026-08-17, once flagged, this was

deliberately reverted rather than left as-is: the fallback is `prefix_fy_seq` again, so a fresh, never-configured business's first invoice numbers `SH-2026-27-0001` -style, matching what every piece of this app's documentation has always said, unless an admin explicitly picks a different layout in Settings → Document Numbering (see `DEVELOPMENT_LOG.md` Era 11). All numbering examples in this Bible reflect that current, reverted-to-default state.

Benefits

- **Accuracy** — GST, discounts, and stock arithmetic are computed once, in one place, the same way every time (never re-typed by hand).
 - **Accountability** — every create/edit/delete is attributed to a named user and time-stamped in the activity log.
 - **Safety net** — soft-delete and the recycle bin mean mistakes are recoverable, not catastrophic, with extra protection for the documents that legally must not silently vanish.
 - **Compliance-readiness** — the GST filing package is built directly from the same transactional data used to run the business, so there is one source of truth rather than a reconciliation exercise at tax time.
 - **Access control that matches how a small business is actually staffed** — an owner can hand out narrowly-scoped, read-only, or fully-privileged access without needing separate software.
-

2. High-Level System Overview

At the simplest level, Science Hub works like almost every modern web application: a person uses a web browser, that browser talks to a server over the internet, the server checks a database, and sends the answer back to be displayed. Below is that same idea, specific to how this application is actually built.

1. User opens the Science Hub website in their browser (e.g. on Vercel).
2. The browser downloads the page's React code (built with Next.js).
3. The page checks: is this user logged in? (NextAuth session cookie)
4. The page calls an internal API (e.g. `/api/invoices`) to fetch or save data.
5. The API route checks permissions, validates the request, and talks to the database via Prisma.
6. The database (PostgreSQL, hosted on Neon) reads or writes the requested rows.
7. The API sends a JSON response back to the browser.
8. The page updates on screen without a full reload, and its local cache is refreshed.

Everything the user sees is rendered "client-side" — every page in this application is explicitly marked `"use client"`, meaning the browser itself does the work of building the visible page after fetching data, rather than the server pre-building the whole page. This remains a deliberate architectural choice, made to avoid a specific bug pattern this version of Next.js has with server-rendered/cached data (`"use cache"` and `cacheComponents: true` are both confirmed, as of this edition, to still be forbidden in this codebase — see Chapter 3).

The three big layers

Layer	Technology	Responsibility
Presentation (Frontend)	Next.js 16 (App Router) + React 19	Everything the user sees and clicks — forms, tables, dashboards, PDFs on screen.
Application (Backend)	Next.js API Routes (same project)	Business rules: validation, GST math, document numbering, the stock ledger, permission checks, email sending.
Data (Database)	PostgreSQL on Neon, accessed via Prisma ORM	The permanent record of every customer, product, invoice, bill, rate list, and user.

Note: because both the frontend and backend live in the same Next.js project, there is no separate "backend server" to deploy — Vercel builds and hosts both together as one unit.

3. Complete Technology Stack

Every technology below is confirmed present in `package.json` or the source code directly — nothing in this list is assumed.

Core Framework

Technology	What it is (plain English)	Why it's used here
Next.js 16	A framework built on top of React that handles routing (turning URLs into pages), bundling, and server/API code, all in one project.	Lets one project serve both the web pages and the API that powers them, deployed as a single unit on Vercel.
React 19	A JavaScript library for building interactive user interfaces out of reusable "components" (buttons, forms, tables).	Every screen in Science Hub — invoice forms, rate-list builders, dashboards, tables — is built from React components.
TypeScript	A version of JavaScript that adds type-checking, catching a class of bugs (e.g. passing a number where text was expected) before the code even runs.	Used throughout the codebase for safer, self-documenting code.

Data & Storage

Technology	What it is	Where/why it's used
PostgreSQL	A mature, industry-standard relational database — data is stored in structured tables with relationships between them (e.g. an invoice belongs to a customer).	The single source of truth for every business record in Science Hub.
Neon	A cloud hosting service that runs PostgreSQL for you, so nobody has to manage a physical database server.	Hosts the production database; connected via a pooled connection string for efficiency under Vercel's serverless model.
Prisma	An "ORM" (Object-Relational Mapper) — a tool that lets developers write database queries as regular code instead of raw SQL, and keeps the database structure in sync with a single schema file.	<code>prisma/schema.prisma</code> defines all 21 data models; almost every API route uses the generated Prisma client directly to read/write data.
Vercel Blob	A cloud file-storage service for storing uploaded files.	Stores purchase-bill attachments (scanned bills/photos) and the uploaded business logo.

Update: the schema now contains **21 models**, not 19 — `RateList` and `RateListItem` were added (see Chapter 8). `package.json`'s `build` script also changed from a plain `next build` to `prisma migrate deploy && next build:three` fields (`Invoice.balanceDue`, `PurchaseBill.balanceDue`, `Product.isLowStock`) are real Postgres `GENERATED ALWAYS AS (...)` `STORED` columns now, and `prisma db push`'s schema-diffing permanently conflicts with a true stored-generated column — migrations must be applied via committed migration files (`prisma migrate deploy`), never `db push`, from this point forward.

Authentication & Security

Technology	What it is	Where/why it's used
NextAuth v4	A ready-made authentication library that handles login sessions so the team doesn't have to build login/cookie/token handling from scratch.	Configured in <code>src/lib/auth.ts</code> with an email+password ("Credentials") login and JWT sessions lasting 8 hours.
bcryptjs	A one-way password-hashing algorithm — it turns a password into a scrambled string that cannot practically be reversed.	Used to hash every stored user password (bcrypt cost factor 12).
Node's built-in <code>crypto</code> module (AES-256-GCM)	A strong, industry-standard symmetric encryption algorithm.	Encrypts secrets at rest: the Gmail app password and the bank account number, in <code>src/lib/crypto.ts</code> . Can now use a dedicated <code>ENCRYPTION_KEY</code> env var independent of <code>NEXTAUTH_SECRET</code> (both old and new encrypted values remain readable).

Communication & Documents

Technology	What it is	Where/why it's used
Nodemailer	A Node.js library for sending email.	Sends invoice PDFs, and now rate-list PDFs, to customers via Gmail SMTP with an "App Password".
jsPDF	A JavaScript library that can build a PDF file entirely inside the browser.	Assembles the final downloadable/emailable invoice, credit-note, and rate-list PDF files, via one shared client-side generator (<code>generateInvoicePdf.ts</code>).
html2canvas	A library that "photographs" a section of a web page and turns it into an image.	Captures the on-screen invoice/rate-list layout so it can be pasted, page by page, into the jsPDF document.
ExcelJS	A library for generating and parsing real <code>.xlsx</code> Excel files.	Powers "Export to Excel" (Credit Notes, Sales Reports, Purchase Reports, Rate Lists) via a shared <code>/api/export-xlsx</code> endpoint, and now also powers reading an uploaded <code>.xlsx</code> rate-list file back into rows (<code>/api/rate-lists/parse-import</code>) — its first use as an import parser, not just an export writer.
JSZip	A library for building <code>.zip</code> archive files in code.	Bundles the GST filing package (multiple report files) into one downloadable ZIP.

Styling & UX

Technology	What it is	Where/why it's used
CSS Modules	A way of writing CSS so that styles are automatically scoped to one component, preventing accidental style clashes between pages.	Every page/component has its own <code>*.module.css</code> file.
CSS Custom Properties (variables)	Reusable named values in CSS that can be swapped instantly.	Defined in <code>src/app/globals.css</code> ; switching the whole app between light/dark mode, or changing the accent colour, is just swapping variable values.

Hosting & Deployment

Technology	What it is	Where/why it's used
Vercel	A cloud hosting platform purpose-built for Next.js applications; automatically builds and deploys the app whenever code is pushed.	Hosts the entire live application.
Environment Variables	Configuration values (secrets, URLs) kept outside the code itself.	Used for the database connection string, authentication secret, encryption key, Gmail credentials, and the app's public URL. Full list in Chapter 23.
npm	Node.js's package manager.	Drives every install/build/dev script (see <code>package.json</code>). <code>postinstall</code> runs <code>prisma generate</code> and installs the repo's git hooks — never remove it.

Not present in this codebase: there is still no Redis (rate limiting remains an in-memory fixed-window counter, evaluated against Redis/Upstash and deliberately reverted), no Railway hosting, and no AI/Gemini bill-extraction feature.

4. Folder Structure

All application code lives under `src/`. Here is what each major folder is for, and why it exists.

```
src/
  app/
    (dashboard)/      A route "group" – every page a logged-in user sees, sharing one sidebar
    dashboard/        The home KPI dashboard
    sales/             Customers, Invoices, Payments Received, Credit Notes, Rate Lists, Sales Overview
    purchases/         Vendors, Purchase Bills, Payments Made, Purchases Overview
    products/, brands/, categories/  The product catalogue
    reports/           Sales Reports, Purchase Reports, GST Reports
    admin/             User management, activity log, section-permissions manager
    bin/               The Recycle Bin
    settings/          Business identity, bank details, Gmail, logo, terms & conditions, document numbering
  api/                Every backend endpoint – one route.ts file per URL path (see Chapter 13)
  login/, forgot-password/, reset-password/, find-email/      Public, unauthenticated pages
  layout.tsx          The root page shell – fonts, branding, the anti-flicker theme script
  providers.tsx        Wraps every page in Session / Branding / Theme / Toast context
  components/
    ui/               Reusable building blocks: Button, Input, Select, UnitCombo, Badge, Toast, Pagination, PdfF
    layout/           DashboardShell (the real sidebar/topbar/auth-guard), GlobalSearch, Breadcrumb
    dialogs/          Modal (the single app-wide popup), ConfirmDialog, PdfCopyDialog
    invoices/          InvoiceLineItemsCard, InvoiceOptionsRow – shared invoice-form building blocks
    rateLists/         RateListFormBody, RateListItemsTable, RateListPrintArea – shared rate-list building blocks
  lib/               Shared backend/frontend logic (not pages, not UI)
    auth.ts           Login/session configuration
    apiAuth.ts        The four permission-check "guards" every API route uses
    prisma.ts         The single shared database connection
    validation.ts     Every form/field validation rule, shared by client and server
    stockMovement.ts  The stock ledger engine
    invoiceCalc.ts, purchaseBillForm.ts, roundOff.ts          GST/discount/rounding math
    documentNumbering.ts  The configurable number-layout engine + Indian Financial Year helpers (new)
    rateListForm.ts, rateListQuery.ts, rateListImport.ts      Rate List calc/validation, pagination filters, bulk-import
    crypto.ts          Encrypts sensitive settings fields at rest
    activity.ts        Writes to the audit log
    rateLimit.ts       Basic abuse-prevention counters
    generateInvoicePdf.ts  Builds the invoice/credit-note/rate-list PDF
    pdfCache.ts        Caches a generated PDF blob, keyed on the source record's updatedAt (new)
    sections.ts        Defines the six permission "sections" and what they gate
    businessBranding.tsx  Shares the business name/logo across every page
    gstFiling.ts, gstFilingZip.ts, gstFilingWorkbook.ts      Builds the GST filing package and its ZIP/Excel export
  types/
    next-auth.d.ts     Extends the login session's shape with role/id/sections
  prisma/
    schema.prisma      The single source of truth for the entire database structure (21 models)
    seed.ts            Fills a fresh database with realistic demo data for development
```

This structure follows a simple rule: pages live under `app/(dashboard)/.../page.tsx`, the API they talk to lives at the matching path under `app/api/.../route.ts`, and any logic shared by more than one place is pulled out into `lib/`. There is deliberately no separate "controllers/services/repositories" layering — most API routes talk to Prisma directly rather than going through an extra abstraction layer, the established convention for a small, single-tenant application.

What's new here since the previous edition:

- `src/app/(dashboard)/sales/rate-lists/` (`page.tsx`, `new/page.tsx`, `[id]/page.tsx`, `[id]/edit/page.tsx`) — the new standalone Rate List builder, sitting under Sales in the sidebar alongside Invoices and Credit Notes even though it has no database relationship to `Product`.
 - `src/app/api/rate-lists/` (`list/create + [id] get/edit/delete`), `src/app/api/send-rate-list/` (email a rate-list PDF), and `src/app/api/rate-lists/parse-import/` (server-side Excel/CSV parsing for bulk-import).
 - `src/components/rateLists/` — a new component folder (`RateListFormBody.tsx` mirrors `PurchaseBillFormBody` 's two-column layout, `RateListItemsTable.tsx`, `RateListPrintArea.tsx` — the shared PDF markup, reused by both the detail page and the list page's on-demand Preview button).
 - `src/components/ui/UnitCombo.tsx` — a typeable unit combobox (free text + filtered suggestions) that replaced a plain `<Select>` for unit fields app-wide. One deliberate exception: the Rate List items table still uses a plain `Input` for its unit column, because that table's horizontally-scrolling wrapper (`overflow-x: auto`) clips `UnitCombo` 's absolutely-positioned dropdown before it can render — confirmed broken and reverted the same day it was tried.
 - `src/lib/documentNumbering.ts` — new; the shared engine behind the configurable invoice/purchase-bill/credit-note number layouts (Chapter 8/13 have the mechanics).
 - `src/lib/rateListForm.ts`, `rateListQuery.ts`, `rateListImport.ts` — Rate List calculation/validation, list-page filter/sort building, and the shared bulk-import parser (used by both the client-side "paste from Excel" path and the server-side `.xlsx` / `.csv` upload path, so there is exactly one parsing implementation, not two).
 - `src/app/api/pincode-lookup/[code]/` — a small new endpoint (any authenticated user, not admin-only) that proxies India Post's public pincode directory, mirroring the existing admin-only IFSC lookup, so customer/vendor/settings address forms can autofill city/state from a PIN code.
 - `src/components/dialogs/Modal.tsx` gained a `variant="fullscreen"` mode and a `footer` prop, and is now the one popup-dialog component used app-wide (19+ call sites) — see Chapter 5.
 - Edit-page routes are consistently `[id]/edit` everywhere, including the new `sales/rate-lists/[id]/edit` — confirmed on disk (`products/[id]/edit`, `purchases/bills/[id]/edit`, `purchases/vendors/[id]/edit`, `sales/customers/[id]/edit`, `sales/invoices/[id]/edit`, `sales/rate-lists/[id]/edit`); there is no lingering `edit/[id]` route anywhere in the app.
-

5. Application Flow

This is the life of a single, typical user session, beginning to end.

1. Visit the site — redirected to `/login` if no valid session.
2. Enter email + password — NextAuth verifies against the hashed password in the database.
3. A signed session cookie (JWT) is set in the browser, valid for 8 hours.
4. Redirected to the Dashboard — KPI cards load from `/api/reports?type=combined-dashboard`.
5. User navigates via the sidebar (only sees items their role/section grants allow).
6. e.g. Create Invoice → fill form → client validates → submit to `/api/invoices` (POST).
7. Server re-validates everything, computes GST, decrements stock, writes the invoice — all inside one database transaction.
8. Invoice detail page can generate a PDF (client-side), download it, print it, or email it to the customer.
9. Every one of the above write actions is recorded in the Activity Log, visible to admins.
10. Deleting anything moves it to the Recycle Bin (soft-delete) rather than erasing it — invoices/purchase bills/credit notes stay there indefinitely, everything else for 30 days.

Behind every one of those steps, the same repeating pattern is used: the browser calls a `useFetch()` hook pointing at an API route; the API route checks the session, checks the role/permissions, validates the input, talks to Prisma, and returns JSON; the page then updates its own in-memory cache so every open tab/component sees the fresh data without a full page reload.

Two flow-level conventions that changed since the previous edition:

- **Every list screen is now paginated server-side.** Invoices, products, brands, categories, vendors, purchase bills, purchase-bill payments, sales payments, credit notes, and rate lists all fetch `{data, total}` a page at a time rather than the whole matching set, with the search box debounced before it hits the API and a matching `/stats` endpoint supplying the summary totals a single page of rows can no longer produce correctly on its own. A page change or filter change shows a dimmed table (`FloatingSpinner`) instead of dropping back to a full skeleton, and scrolls the list back into view.
- **Every popup dialog in the app is the same component.** Where the previous edition's flow diagrams implicitly assumed a mix of inline expanding sections and one-off modals (e.g. Settings' sections used to expand in place and auto-scroll on mobile), every popup — Settings' six sections, both invoice/purchase-bill quick-add-item modals, inline vendor/customer creation, Categories/Brands Add & Rename, Admin's New/Edit User, and both Rate List modals — now opens as the same `Modal` component in `variant="fullscreen"` mode, with Save/Cancel pinned in a sticky footer outside the scrollable body rather than living inside the form itself.

A Rate List follows a lighter version of the same invoice-style flow: build the sheet (typed rows, or bulk-imported via paste or file upload, always still reviewable/editable before saving) → save → the detail page can Preview/Download/Regenerate the PDF, Email it to an address typed in at share-time (it has no linked customer to default one from), or Export to Excel — but it never touches stock or the ledger, since it is deliberately unconnected to the Product catalogue.

6. User Roles & Permissions

Science Hub has three roles, stored as a plain text field (`User.role`) — there is no rigid, database-enforced list of allowed roles; the application code is what enforces which three values are meaningful: `admin`, `staff` (the default for new users), and `manager`. This three-role model, and the section-permission system below, are unchanged in shape since the previous edition and remain verified accurate against both `src/lib/apiAuth.ts` and `src/lib/sections.ts`.

6.1 Admin

`ADMIN` has unrestricted access to everything. Only an admin can reach: Settings (business identity, bank details, Gmail credentials, logo, **document numbering**), Admin (user management, activity log, section-permissions manager), permanently delete anything from the Recycle Bin, run the IFSC bank-code lookup, and bulk-clear ledger history. An admin automatically passes every section-permission check — nothing needs to be individually granted to an admin account, and the permissions API explicitly refuses to let anyone set section grants on an admin account.

6.2 Staff (default role)

`STAFF` can fully create, edit, and delete the core business records — customers, products, invoices, vendors, purchase bills, brands, categories, **rate lists** — without restriction. What is restricted for staff is visibility into six specific "sections" of the app (dashboards, reports, and payment-history pages) — see 6.4 below. By default a brand-new staff account has none of these six sections enabled and must be granted them by an admin.

6.3 Manager — read-only

`MANAGER` is functionally identical to staff for the purpose of section-based visibility, but is hard-blocked from every mutating action in the system — creating, editing, deleting, recording a payment, cancelling a bill, restoring from the bin — all of it. This is enforced in exactly one shared rule (`requireWriteAccess()` on the server, mirrored by a `useCanWrite()` check on the client) rather than being re-implemented per page, so it is applied consistently everywhere. Managers also cannot see the Recycle Bin link in navigation at all.

6.4 Section-based permissions (an additional, finer-grained layer)

Independently of the three roles above, an admin can grant or revoke access to exactly six "sections" on a per-user basis, for any non-admin user (staff or manager). Confirmed directly against `src/lib/sections.ts` — the set is unchanged since the previous edition:

Section key	What it controls
<code>sales_overview</code>	The Sales Overview dashboard and its KPI widgets on the main Dashboard page
<code>purchase_overview</code>	The Purchases Overview dashboard and its KPI widgets on the main Dashboard page
<code>reports_sales</code>	The Sales Reports page
<code>reports_purchases</code>	The Purchase Reports page
<code>payments_received</code>	The Payments Received history page

Section key	What it controls
<code>payments_made</code>	The Payments Made history page

Two sections are combined for a seventh, stricter gate: the GST Reports page and its underlying API require **both** `reports_sales` and `reports_purchases` together (or admin) — a deliberate design decision so a combined tax-filing report is never handed to someone with only half the relevant visibility.

This is managed from Admin → Permissions, a grid where each row is a grantable user and each column is one of the six sections, with a simple on/off toggle per cell. There is no separate "revoke" action — toggling a cell off is the same operation as toggling it on.

6.5 Permission enforcement summary

Check	Enforced where	Effect
<code>requireSession()</code>	Nearly every API route	401 if not logged in at all
<code>requireAdmin()</code>	User management, permissions, settings writes, bin permanent-delete, stock-ledger bulk-clear	403 unless role is admin
<code>requireWriteAccess()</code>	Every create/edit/delete endpoint for core business data (invoices, rate lists included)	403 if role is manager
<code>requireSectionAccess(section)</code>	Overview dashboards, reports, payment-history APIs	403 unless admin or the section is individually granted

All of the above are enforced on the server (`src/lib/apiAuth.ts`), independently of the sidebar/page-level redirects that hide navigation items or bounce a manager away from an edit form — those are a convenience layer only.

Update: `middleware.ts`'s public allowlist was re-verified directly against the source for this edition. `PUBLIC_API_PREFIXES` is `["/api/auth/", "/api/setup"]` and `PUBLIC_API_EXACT` is `={"/api/settings/branding"}` — nothing else is exempt from the default-deny session check. In particular, `/api/send-rate-list` (the new rate-list email endpoint) is **not** on this list and correctly requires a session plus `requireWriteAccess()` inside its own handler, exactly like `/api/send-invoice` — new routes added since the previous edition did not silently widen the public surface.

7. Authentication & Sessions

7.1 How login works

1. User submits email + password on `/login`.
2. Email is lowercased/trimmed; NextAuth's "Credentials" provider runs `authorize()`.
3. Rate limits checked: max 30 attempts/15min per IP address, and max 8 attempts/15min per account.
4. Password checked with `bcrypt.compare()` against the stored hash.
5. On success: a signed JWT is issued containing user id, role, and granted sections.
6. JWT is stored in an `httpOnly`, secure, 8-hour session cookie.

A subtle but important detail: whether the email doesn't exist or the password is wrong, the server always still runs a `bcrypt` comparison (against a fixed dummy hash if no such user exists) before responding, so both failure cases take roughly the same amount of time — the standard defence against a timing attack used to enumerate valid emails.

Update (2026-08-17): `authorize()` now wraps the `prisma.user.findUnique()` lookup in its own `try/catch`. Previously, a transient database error during login (a Neon cold-start, pooled-connection contention under `connection_limit=1`, or a brief network blip) would throw out of `authorize()` uncaught; NextAuth already collapses any thrown error into the same generic "Incorrect email or password" response shown to the user either way, so this fix changes nothing about what the user sees — its entire purpose is that the real cause now gets logged server-side (`console.error("[auth] DB lookup failed during login attempt", ...)`) instead of being swallowed silently, so a spike in failed logins caused by a database blip can actually be diagnosed rather than looking identical to a spike in wrong passwords.

7.2 What's inside the session

The session available to every page (via `useSession()`) and every API route (via `getSession()`) contains: the user's `id`, `name`, `email`, `role`, and `sections` (the array of section keys they've been granted). Section grants are reloaded into the token every 30 seconds; a full account-validity check (has the password changed / has `tokenVersion` been bumped) happens at most every 5 minutes, so an admin revoking access or changing a role takes effect quickly without forcing every user to log out and back in. A JWT issued before `tokenVersion` existed on the token is backfilled rather than invalidated, so shipping that check never signs out an already-logged-in user by accident.

7.3 Cookies and JWT — the technical detail, explained simply

A JWT (JSON Web Token) is a small, tamper-proof, signed piece of text that encodes who the logged-in user is. Because it's signed with a secret only the server knows, the browser can carry it around in a cookie and the server can trust its contents without needing to look the user up in the database on every single request. The cookie itself is marked `httpOnly` (JavaScript running in the page can never read it) and `Secure` in production (only ever sent over HTTPS).

7.4 Password reset flow

1. User clicks "Forgot Password" → enters their email.
2. Server always replies "check your email" — whether or not that email exists.
3. If the account is real: a random, one-time 1-hour token is generated and emailed as a reset link.
4. User clicks the link, enters a new password (minimum 8 characters).
5. Server checks the token hasn't already been used and hasn't expired, then updates the password and immediately invalidates the old login session.

There is also a separate "Find my email" page: a user can search by their name and get back a masked version of their registered email (e.g. `jo***@example.com`) without ever exposing the full address or whether a name exists in the system with certainty.

7.5 Password storage

Passwords are never stored in readable form. Every password is hashed with bcrypt at a cost factor of 12 before being saved — even someone with direct database access cannot read anyone's actual password.

7.6 Authorization vs. authentication

Authentication answers "who is this?" (the login/session system above). Authorization answers "what is this person allowed to do?" — handled by the four guard functions described in Chapter 6.5. Every API route in this codebase that touches non-public data calls at least one of these guards before doing anything else.

7.7 The middleware gatekeeper

A `middleware.ts` file at the project root runs in front of every request to `/api/**` and enforces a default-deny baseline: unless the path is on the explicit public allowlist (`/api/auth/*`, `/api/setup`, `/api/settings/branding` — re-verified for this edition, see Chapter 6.5's Update box), the request must carry a valid session token or it is rejected with 401 before it ever reaches a route handler. This does not replace the per-route guards described in Chapter 6.5 — those still enforce the more specific role/section rules — it exists purely as a safety net so a brand-new API route is protected automatically even if a developer forgets to add a guard call to it.

8. Database (Full ER Reference)

The database has **21 tables** (called "models" in Prisma) — up from 19 in the previous edition — defined in one file: `prisma/schema.prisma`. There are no database-level enum types in this schema — anything that looks like a fixed set of choices (like an invoice's status) is a plain text field whose allowed values are enforced by the application code, not the database itself.

8.1 Soft-delete vs. permanent records

Has a "Recycle Bin" (<code>deletedAt</code> field)	Never soft-deleted (permanent/append-only)
Customer, Category, Brand, Product, Invoice, Return (credit note), Vendor, PurchaseBill, RateList	User, PasswordResetToken, ActivityLog, InvoiceItem, Payment, ReturnItem, BusinessSettings, PurchaseBillItem, PurchasePayment, StockMovement, SectionPermission, RateListItem

Update: `RateList` gained a `deletedAt` field, joining the soft-delete side of this table for the first time this edition. Its child rows (`RateListItem`) are cascade-deleted with it, same pattern as `InvoiceItem` / `Invoice`. Unlike every other soft-deletable type, `RateList` currently has **no restore UI from the Recycle Bin** — `DELETE /api/rate-lists/[id]` sets `deletedAt`, but the Bin page's 8 entity types (invoices, purchase bills, credit notes, customers, products, brands, categories, vendors) do not yet include rate lists, so a soft-deleted rate list is currently unrecoverable from the UI even though the underlying row survives in the database. This is a known, deliberately-scoped gap (see Pending Tasks in the project's own `CLAUDE.md`), not an oversight in this documentation.

The rule of thumb otherwise: anything a business person might reasonably want to "undo" deleting (a customer, a product, an invoice, a rate list) is soft-deleted. Line items, payments, and log entries live and die with their parent record, and the ledger/log tables are never deleted at all in normal operation.

8.2 Every table, field by field

User — every person who can log in

`id`, `name`, `email` (unique), `password` (bcrypt hash), `role` (default "staff"), `tokenVersion` (Int, bumped to invalidate sessions on password change), `createdAt` Relations: owns many Invoices, ActivityLogs, PasswordResetTokens, PurchaseBills (as creator), StockMovements (as creator), SectionPermissions, and — new this edition — **RateLists** (as creator).

PasswordResetToken

`id`, `userId` → User, `token` (unique, random 256-bit), `expiresAt`, `usedAt` (marks it spent), `createdAt` Deleting a User cascades to delete their reset tokens.

ActivityLog — the audit trail

`id`, `userId` → User, `action` (e.g. "create_invoice"), `details` (free text), `entityId?`, `entityType?`, `createdAt` Indexed by `userId` and by `createdAt` for fast filtering in the Admin activity page.

Customer

id, name, phone?, email?, address?, city?, state?, pincode?, gstin?, createdAt, updatedAt, deletedAt? Has many Invoices. Indexed on deletedAt .

Category / Brand — catalogue grouping

id, name (unique), createdAt, updatedAt, deletedAt? Each has many Products. Indexed on deletedAt .

Product

id, name, description?, sku? (unique), barcode?, hsn?, unit (default "Nos"), price (sale price), purchasePrice?, gstRate (default 18), stock (default 0), minStock (default 5), maxStock?, reorderLevel?, categoryId?, brandId?, isActive (default true), createdAt, updatedAt, deletedAt? **isLowStock** — a real Postgres GENERATED ALWAYS AS (stock > 0 AND stock <= "minStock") STORED column, not a plain application-computed field; it exists so a cross-column comparison ("is stock above zero but at or below the minimum?") can be filtered server-side with a normal boolean equality check instead of raw SQL, and mirrors src/lib/stockStatus.ts's isLowStock() definition exactly. Never written to by application code. Belongs to one Brand and one Category (both optional). Referenced by InvoiceItems, ReturnItems, PurchaseBillItems, and StockMovements. Indexed on deletedAt , categoryId , brandId .

Invoice — the sales bill

id, invoiceNumber (unique), date, dueDate?, customerId → Customer, userId → User (creator), status (unpaid/partial/paid), subtotal, cgst, sgst, igst, roundOff, total, paidAmount, notes?, isInterState, placeOfSupply?, reverseCharge, createdAt, updatedAt, deletedAt? **balanceDue** — a real Postgres GENERATED ALWAYS AS (total - "paidAmount") STORED column, letting the invoices list sort server-side by outstanding balance without raw SQL. Never written to by application code. **New fields this edition — transportation charge:** transportCharge (Float, default 0), transportChargeGstRate (Float, default 0), transportChargeGstAmount (Float, default 0) — an optional freight/transport line shown on its own row on the printed invoice, with its own GST rate and amount, kept deliberately separate from cgst / sgst / igst (which stay a pure sum of item-level tax only) rather than folded into the inter/intra-state split. transportChargeGstAmount is always server-recomputed from transportCharge × transportChargeGstRate, never trusted from the client, matching the same client-doesn't-get-the-final-say pattern used for isInterState . Has many InvoiceItems, Payments, and Returns (credit notes). Indexed on deletedAt , status , customerId , date .

InvoiceItem

id, invoiceId (cascade delete), productId?, name, hsn (default ""), quantity, unit, price, discountPercent (default 0), discountAmount (default 0), gstRate, gstAmount, total Indexed on invoiceId , productId .

Payment — money received against an invoice

id, invoiceId (cascade), amount, method (default "cash"), reference?, date, notes? Indexed on invoiceId .

Return / ReturnItem — a credit note

Return: id, invoiceId (cascade), creditNoteNumber? (unique – nullable so pre-existing returns aren't retroactively renumbered), date, notes?, subtotal, cgst, sgst, igst, roundOff, total, createdAt, deletedAt? ReturnItem: id, returnId (cascade), productId?, name, quantity, price, gstRate (default 0), gstAmount (default 0), total Return indexed on invoiceId , deletedAt .

BusinessSettings — one single row, id="singleton"

name (default "Science Hub"), tagline, logoUrl, email, phone, address, city, state, pincode, gstin, pan, gmailUser, gmailAppPassword (encrypted), bankName, bankAccountName, bankAccountNumber (encrypted), bankIfsc, bankBranch, termsAndConditions, showLogoOnInvoices (default true), updatedAt

New fields this edition — configurable document numbering, one independent set per document type:

Field group	Invoice	Purchase Bill	Credit Note
Custom prefix override	invoiceNumberPrefix?	purchaseBillNumberPrefix?	creditNoteNumberPrefix?
One-time "next number" override	nextInvoiceNumberOverride?	nextPurchaseBillNumberOverride?	nextCreditNoteNumberOverride?
Layout selection	invoiceNumberFormat?	purchaseBillNumberFormat?	creditNoteNumberFormat?

All nine fields are nullable — `null` means "use the auto default" for that field. The "next number" override fields are one-time-use: the moment a create picks one up, it is cleared back to `null` in the same transaction so it can never silently re-apply to a later document. See Chapter 13 for the full `NUMBER_FORMATS` layout mechanism and the Indian Financial Year (1 April – 31 March) reset boundary that every layout's year segment follows.

There is exactly one row of this table, ever — the whole application's identity, letterhead, mail-sending credentials, and numbering configuration.

Vendor — a supplier the business buys from

id, name, company?, gstin?, phone?, email?, address?, city?, state?, pincode?, notes?, isActive (default true), createdAt, updatedAt, deletedAt? Has many PurchaseBills.

Update: Vendor gained `city?`, `state?`, and `pincode?` fields since the previous edition (which listed only a single free-text `address?`) — these back the same pincode-autofill flow now used on the customer/settings address forms (`/api/pincode-lookup`).

PurchaseBill

id, billNumber (unique), vendorId → Vendor, billDate, dueDate?, subtotal, taxAmount, discount, roundOff, total, paidAmount, status (unpaid/partial/paid/cancelled), notes?, attachmentUrl?, attachmentName?, category? (free text), createdByUserId → User, createdAt, updatedAt, deletedAt? **balanceDue** — same `GENERATED ALWAYS AS (total - "paidAmount") STORED` pattern as `Invoice.balanceDue`, backing the "Balance Due (High-Low)" sort option.

Update: PurchaseBill now carries its own GST split fields — `isInterState` (default false), `placeOfSupply?`, `cgst`, `sgst`, `igst` (all `Float`, default 0) — which the previous edition's schema listing did not show. Purchase bills compute and store inter/intra-state tax the same way invoices do, not just a single flat `taxAmount`.

New fields this edition — transportation charge, identical shape/reasoning to `Invoice`'s: `transportCharge` (`Float`, default 0), `transportChargeGstRate` (`Float`, default 0), `transportChargeGstAmount` (`Float`, default 0).

Has many PurchaseBillItems, PurchasePayments, and StockMovements. Indexed on `vendorId`, `status`, `billDate`.

PurchaseBillItem / PurchasePayment

PurchaseBillItem: id, purchaseBillId (cascade), productId?, name, hsn (default ""), quantity, unit (default "Nos"), purchasePrice, discountPercent (default 0), discountAmount (default 0), gstRate (default 0), gstAmount (default 0), total PurchasePayment: id, purchaseBillId (cascade), amount, method (default "cash"), reference?, date, notes?, createdAt PurchasePayment indexed on purchaseBillId.

StockMovement — the append-only inventory ledger

id, productId? (SetNull on product delete), productName (a permanent snapshot of the name, so history survives even product deletion), type (a specific reason – see below), documentType (invoice/purchase_bill/credit_note/manual), quantity (signed: positive = stock in, negative = stock out), balanceAfter, reference?, notes?, purchaseBillId?, createdByUserId?, createdAt Never updated after creation — a true, permanent audit trail. Indexed on [productId, createdAt], createdAt, documentType.

RateList — new this edition; a standalone price-sheet/catalogue builder, deliberately independent of the Product catalogue (a rate list's brand/unit strings, e.g. "QUALIGENS" / "500 GM", routinely don't match this app's own Brand / Product records)

id, title, note?, createdByUserId → User, createdAt, updatedAt, deletedAt? Has many RateListItems (onDelete: Cascade). Indexed on deletedAt.

RateListItem — new this edition

id, rateListId → RateList (onDelete: Cascade), serialNo (Int – display order), name, brand? (free text, no relation to Brand), unit (free text, no relation to any unit enum), isNetRate (Boolean, default false), discountPercent (Float, default 0 – ignored when isNetRate is true), listRate (Float), amount (Float – computed server-side: $\text{isNetRate} ? \text{listRate} : \text{listRate} - \text{listRate} * \text{discountPercent} / 100$, rounded to 2dp) Indexed on rateListId.

SectionPermission — the fine-grained access grants (Chapter 6.4)

id, userId (cascade), section (one of the six section keys), enabled (default false), createdAt, updatedAt One row per (user, section) pair — enforced by a database-level uniqueness constraint (@@unique([userId, section])), plus an index on userId.

8.3 Key relationships, plain English

- One Customer can have many Invoices. One Invoice belongs to exactly one Customer.
- One Invoice has many InvoiceItems (the line items), many Payments, and many Returns (credit notes issued against it).
- One Vendor can have many PurchaseBills; the same one-to-many pattern repeats for bill items and payments.
- One RateList has many RateListItems, cascade-deleted together — but a RateList itself has **no relationship at all** to Product, Brand, or Category; its items are free text by design.
- A Product can be referenced by invoice line items, return line items, and purchase-bill line items — but if the product is later deleted, only the StockMovement ledger is designed to survive that (its link to the product is set to nothing, while the product's name is preserved forever as a text snapshot).
- Every meaningful action a user takes can be traced back to them via `ActivityLog.userId`, and every stock change can be traced to both the user who caused it and, if relevant, the exact purchase bill that caused it.

Data integrity in practice: deleting an Invoice automatically and safely deletes its InvoiceItems and Payments (enforced by the database itself via "cascade" rules, not just application code) — so it is not possible to end up with

orphaned line items pointing at a non-existent invoice. The same is true of a deleted RateList and its RateListItems.

9. Complete Module Documentation

9.1 Invoices

Purpose: create, edit, and manage GST sales invoices. Numbering is no longer the fixed `SH-{year}-{0001}` shape described in the 2nd edition — as of the Configurable Document Numbering feature (2026-08-09/10, see the new §9.19), the invoice number's **layout**, **prefix**, and a one-time **next-number override** are all admin-configurable per business (Settings → Document Numbering), generated inside a Serializable database transaction with automatic retry on conflict (`P2034`) if two invoices are created at the exact same instant — see Chapter 10 for the full mechanism. The year segment embedded in the number is always the Indian Financial Year (1 April – 31 March), never the calendar year.

Business rules (unchanged from the 2nd edition unless noted): place-of-supply is mandatory on every invoice; a due date cannot be in the past; the same product cannot appear as two separate line items (quantities must be combined); a fully-paid invoice cannot be edited at all. `isInterState` is never trusted from the client — the server independently re-derives it via `deriveIsInterState()` (`src/lib/gstLocation.ts`), comparing place-of-supply against the business's own configured state, falling back to the client-supplied value only if the business's state isn't configured yet.

New since 2nd edition:

- **Transportation charge.** `Invoice.transportCharge` (`Float @default(0)`) is an optional freight/transport line, shown as its own row on the printed invoice with its own GST rate (`transportChargeGstRate`) and GST amount (`transportChargeGstAmount`). It is deliberately kept out of the invoice's `cgst` / `sgst` / `igst` totals — those columns stay a pure sum of item-level tax — rather than being folded into the inter-/intra-state split. `transportChargeGstAmount` is always server-recomputed from `charge × rate` inside `computeInvoiceTotals()` (`src/lib/invoiceCalc.ts`), never trusted from the client; the function signature is `computeInvoiceTotals(items, transportCharge = 0, transportChargeGstRate = 0)`, and the raw total is `subtotal + totalTax + transportCharge + transportChargeGstAmount` before rounding. The New Invoice, Invoice Edit, and PDF renderer all read/write this triplet of fields identically.
- **Editable invoice date, FY-bounded.** An invoice's `date` can be corrected after creation (e.g. a data-entry typo) but the edit is rejected if the new date would fall in a different Indian Financial Year than the one the invoice's number was generated against — since the number's year segment would then disagree with the document's own date. This applies identically to Purchase Bills (§9.7) via their `billDate`.
- **Editable "Bill To" customer from Invoice Edit** (2026-08-something, see CLAUDE.md feature 34). The customer info block on the Invoice Edit page has its own "Edit" link (`openCustomerEdit`) opening a `Modal` with the same fields/validation as the New Invoice/New Purchase Bill inline-customer forms (name/address/pincode+state with IFSC-style autofill/city/GSTIN/phone/email), which `PUT` s `/api/customers/[id]` directly. This lets a mistake in a customer's saved details be fixed without leaving the invoice-edit flow. It deliberately does **not** touch the invoice's own `placeOfSupply` / `isInterState` even if the edited customer's state changes — those are independently editable on the same page, and silently re-deriving them here could change a GST calculation the user didn't intend to touch.

Dependencies: Customers, Products (for stock and GST rate), the stock ledger, the configurable document-numbering system (§9.19), and (optionally) the PDF/email system.

9.2 Customers

Purpose: the address book of who invoices are billed to. Business rule: a customer with any active (non-binned) invoice cannot be soft-deleted — they must be reassigned or their invoices removed first. A customer can also be created "inline" directly from the invoice-creation screen, or (new) edited inline from the Invoice Edit page's "Bill To" block (see §9.1).

9.3 Products, Brands, Categories

Purpose: the catalogue. Each product has a sale price, an optional purchase price, a GST rate, a current stock count, and a minimum-stock threshold used to flag low stock via the single shared definition in `src/lib/stockStatus.ts` (`isOutOfStock` / `isLowStock` / `needsRestock`) — every dashboard, report, and list screen defers to these three functions rather than re-deriving the comparison inline. `Product.isLowStock` itself is a real Postgres `GENERATED ALWAYS AS ("stock" > 0 AND "stock" <= "minStock") STORED` column, mirroring `isLowStock()`; application code never writes to it directly.

Business rule: a product, brand, or category used on any invoice line item or active purchase-bill line item cannot be soft-deleted. Permanently deleting from the bin applies a stricter check (any purchase bill, including ones already in the bin, not just active ones).

The product form's unit-suggestion list (`PRODUCT_UNITS` in `src/lib/productForm.ts`) had one entry renamed on 2026-08-11 — "Pack" → "Pkt" — a cosmetic label change to the suggestion list only; it does not affect stored data for existing products, and the field remains free text regardless.

Note on unit entry generally: product/invoice/purchase-bill unit fields that need a typeable-plus-suggestions UI now use the shared `UnitCombo` component (`src/components/ui/UnitCombo.tsx`, see §9.20) rather than a constrained `Select` — this replaced two near-duplicate hand-rolled combos previously living in the invoice and purchase-bill "Add Custom Item" quick-add modals.

9.4 Payments (Received)

Purpose: record money received against an invoice. Every time a payment is added or edited, the invoice's paid amount is recalculated from the true sum of all its payments (not just incrementally adjusted), and its status (unpaid/partial/paid) is derived fresh from that sum, so it can never silently drift out of sync. The payment-edit route (`PUT /api/invoices/[id]/payment/[paymentId]`) runs inside the same `Serializable-transaction-with-P2034-retry` pattern as payment creation and every other money-mutating flow in the app — this was a gap in the 2nd edition's era, since fixed (see Known Issues in Chapter-level notes).

9.5 Returns / Credit Notes

Purpose: handle a customer returning goods. A return can only be recorded against an invoice that has received at least one payment. It is capped in two independent ways: it cannot refund more than the invoice has actually been paid, and it cannot return more of a product than remains un-returned on that invoice. Every return is a `Return` row and is itself a legally-numbered **credit note** — `Return.creditNoteNumber` (nullable, unique — older rows predate numbering). Returning goods restores the returned quantity back into stock via `recordStockMovement()`.

Expanded since 2nd edition — credit-note numbering is now configurable. Credit notes were previously numbered with the same fixed mechanism as invoices/purchase bills (`CN-{year}-{0001}`). They now go through the identical configurable-layout system described in §9.19: `BusinessSettings.creditNoteNumberPrefix` (default "CN"), `nextCreditNoteNumberOverride` (one-time-use), and `creditNoteNumberFormat` (one of the three `NUMBER_FORMATS`

layouts, independent of whatever layout invoices or purchase bills use) — an admin can, for example, run invoices on `prefix_fy_seq` while credit notes stay on the plain `seq_fy` style, or vice versa. The year segment is the credit note's own Indian Financial Year, matching its parent invoice's date-vs-FY constraints.

Credit Notes list page (`sales/credit-notes`) is a dedicated view over every non-deleted `Return` row across all invoices — not just the returns visible from one invoice's detail page — with its own search/sort/pagination (`buildCreditNoteWhere` / `buildCreditNoteOrderBy` in `src/lib/creditNoteQuery.ts`, `{data, total}` shape, `GET /api/credit-notes` + a companion `GET /api/credit-notes/stats` for summary totals independent of the current page), PDF, and `.xlsx` export via the shared `/api/export-xlsx` infrastructure (see §9.17).

Credit notes are one of the three document types exempt from the Bin's 30-day auto-purge — see §9.13.

9.6 Vendors

Purpose: the suppliers the business buys from. Creating a vendor requires phone and address; editing one does not (loosened, since existing vendor records may predate that requirement). A vendor with any active purchase bill cannot be soft-deleted.

9.7 Purchase Bills

Purpose: record a bill from a vendor. Numbered using the same configurable-layout mechanism as invoices (§9.19) via `BusinessSettings.purchaseBillNumberPrefix` / `nextPurchaseBillNumberOverride` / `purchaseBillNumberFormat` (default prefix `"PB"`), with the Financial Year segment derived from the bill's own `billDate` — not `"now"` — since a bill can legitimately be entered late for an earlier period. Supports an optional file attachment (a photo or PDF of the physical bill, magic-byte validated, Vercel Blob), an optional inline first payment at creation time, and a Cancel/Un-cancel workflow distinct from deletion — cancelling reverses the stock the bill added without removing the record; deleting soft-deletes it entirely and does not double-reverse stock if it was already cancelled.

New since 2nd edition: `PurchaseBill.transportCharge` / `transportChargeGstRate` / `transportChargeGstAmount` — the identical shape and reasoning as `Invoice.transportCharge` (§9.1): its own line, its own GST rate/amount, kept separate from `cgst` / `sgst` / `igst`, and `transportChargeGstAmount` is always server-recomputed, never trusted from the client. `billDate` is editable post-creation but blocked from crossing a Financial Year boundary, same rule as invoices.

9.8 Payments (Made)

Purpose: record money paid to a vendor against a purchase bill. Mirrors the sales-side payment logic (balance re-checked inside a database transaction so two simultaneous payments cannot together overpay a bill).

9.9 Stock & the Stock Movement Ledger

Purpose: the single, permanent, append-only history of every reason a product's stock quantity ever changed. Detailed fully in Chapter 11. `StockMovement.type` is a closed, specific union (`sale`, `sale_edit_reverse`, `sale_edit_apply`, `sale_delete_restore`, `sale_bin_restore`, `purchase`, `purchase_edit_reverse`, `purchase_edit_apply`, `purchase_cancel`, `purchase_uncancel`, `purchase_delete_restore`, `purchase_bin_restore`, `return`, `return_delete_reverse`, `return_bin_restore`, `manual`) — there is deliberately no generic `"adjustment"` catch-all; every stock-affecting code path must name the most specific type that applies.

9.10 Reports

Purpose: summarised views for decision-making — this month's revenue/spend, outstanding balances (with an "aging" breakdown of how overdue a purchase bill is), a 12-month GST summary, and management dashboards. Detailed in Chapter 13's API reference. The Purchase Dashboard's Top Vendors and monthly spend/paid aggregates exclude cancelled bills, consistent with the Purchase Bills module's cancel/un-cancel workflow (§9.7).

9.11 GST Filing

Purpose: assembles a ready-to-file GST return package — a Sales Register, a B2B/B2C split, a Credit Notes list, a Purchase Register, and an HSN-code summary — for a chosen month or financial year, downloadable as a ZIP for handing to an accountant. Gated by `requireGstFilingAccess()` so that only an admin, or a user granted both the `reports_sales` and `reports_purchases` sections, can generate it.

9.12 Settings

Purpose: the business's own identity — name, tagline, logo, GSTIN/PAN, bank details (for printing on invoices), the Gmail account used to send invoice/rate-list emails, default Terms & Conditions text, and (new, §9.19) the Document Numbering configuration for invoices, purchase bills, and credit notes. Editable only by an admin.

Changed since 2nd edition: the settings page's editing UX was reworked (2026-08-14) from inline expanding sections — which used to push the surrounding page layout and, on mobile, auto-scroll awkwardly to the newly-expanded block — into six independent fullscreen `Modal` popups (Business Identity, Address, Bank Details, Terms & Conditions, Email, Document Numbering). Each section still saves independently: editing bank details can never accidentally wipe the Gmail configuration or vice versa, exactly as in the 2nd edition, just via a different UI container. See §9.21 for the shared `Modal` component this depends on.

9.13 Recycle Bin

Purpose: a safety net for 7 soft-deletable record types (invoices, purchase bills, customers, products, brands, categories, vendors) plus credit notes (returns) as an 8th category surfaced in the bin listing. `GET /api/bin` auto-purges most types older than 30 days on every load, each with its own FK-safety check (e.g. skipping a customer still referenced by an active invoice) and annotates surviving rows with `protectedReason`. An admin can permanently delete one item at a time from the bin, or (admin-only) empty the entire bin at once via `DELETE /api/bin/empty`, which purges in FK-safe order (invoices/purchase-bills first, then customers/products, then brands/categories, then vendors).

Materially changed since 2nd edition — GST-safe retention for numbered documents (2026-08-11).

Invoices, purchase bills, and credit notes are now **exempt from the 30-day auto-purge**. This closed a compliance gap: under the old blanket 30-day rule, a staff member could delete `PB-2026-27-0025`, create a replacement numbered `...-0026`, and 30 days later have `...-0025` silently vanish from the bin with no way to explain the resulting sequence gap during a GST audit. These three types now stay in the bin indefinitely — `GET /api/bin` reports `daysLeft: -1` for them, rendered in the UI as a green "Retained (GST)" pill instead of a countdown — until an admin explicitly permanently deletes one from the Bin page, which shows an inline GST sequence-gap warning in the confirm dialog before allowing it. This is the *only* way to hard-delete one of these three document types; each entity's own `DELETE` route (`/api/invoices/[id]`, `/api/purchase-bills/[id]`) only ever soft-deletes and accepts no `?permanent=true` override. Emptying the whole bin via `DELETE /api/bin/empty` is the one exception that still purges

all three types immediately, on the reasoning that emptying the bin is itself the deliberate, explicit action that makes hard-deleting them safe to do without a per-item warning.

Rate Lists (§9.18) currently have **no** Bin integration — `DELETE /api/rate-lists/[id]` performs a real `deletedAt` soft-delete, but there is no restore/permanent-delete UI for that row anywhere yet. This is a known, deliberately-scoped gap (see §9.18's "Current Gaps").

9.14 Admin — Users & Activity Log

Purpose: create/edit/delete staff accounts, assign roles (admin/staff/manager), grant/revoke the six `ProtectedSection` permissions per non-admin user (Admin → Permissions, `SectionPermission` rows, admin always bypasses the check), and browse a searchable, paginated log of every meaningful action taken in the system, by whom, and when.

9.15 Global Search

Purpose: one search box in the topbar that simultaneously searches invoices, customers, products, vendors, purchase bills, brands, and categories (five results per category), with results appearing as the user types, debounced 250ms with `AbortController` cancellation. Rate Lists are **not** currently part of Global Search's entity set.

9.16 Email

Purpose: send a document PDF straight to a recipient's inbox using the business's own configured Gmail account (falls back to `EMAIL_USER` / `EMAIL_APP_PASSWORD` env vars if `BusinessSettings` isn't configured). Two send endpoints exist:

- `POST /api/send-invoice` — sends an invoice PDF to the invoice's linked customer email (pre-filled), rate-limited (20/15 min per user).
- `POST /api/send-rate-list` — sends a Rate List PDF (§9.18); since a rate list has no linked customer, the recipient address is always typed in manually at share-time via a small `Modal`. Otherwise mirrors `/api/send-invoice` exactly: same Gmail-credential resolution order, same 20/15-min rate limit, `requireWriteAccess()` - gated, validates the recipient address with a simple email regex, caps the attached PDF at 10 MB, and returns a 503 with a clear message if Gmail isn't configured.

9.17 PDF Generation

Purpose: produce a pixel-accurate, print-ready, GST-compliant PDF entirely inside the browser via `generateInvoicePdfBlob()` (`src/lib/generateInvoicePdf.ts`) — detailed fully in Chapter 17. As of Rate Lists (§9.18), this pipeline is no longer invoice-exclusive: it is reused as-is for Rate List PDFs, because the function's actual page-splitting requirement is only "a `<table>` with `<tbody>` rows" — every invoice-specific feature (repeating `thead` / `tfoot`, `data-invoice-item-row` markers, `#invoice-page-marker`, multi-copy Original/Duplicate stamping) is optional and simply no-ops when the target markup doesn't provide it.

Generated PDFs for invoices, purchase bills, returns, and rate lists are cached client-side in IndexedDB (`src/lib/pdfCache.ts`, `PdfEntity = "invoice" | "purchase-bill" | "return" | "rate-list"`), keyed by entity id plus a variant key covering copy labels and any render-affecting flags (`buildPdfVariantKey()`), so re-opening a detail page's Preview doesn't re-run the expensive `html2canvas + jsPDF` render unless something that affects the output actually changed. The cache is wiped entirely on sign-out and never persists across logins.

9.18 Rate Lists (new module, 2026-08-11)

Purpose: a standalone, downloadable price-sheet/catalog builder for sharing a quotation or product-and-pricing sheet with a customer (e.g. "Chemical Rate List — August 2026"), reachable at `sales/rate-lists` in the SALES nav group.

Deliberately not linked to the Product catalog — a `RateListItem` row's `name` / `brand` / `unit` are free text, since a rate list's own vocabulary (e.g. brand `"QUALIGENS"`, unit `"500 GM"`) routinely doesn't match this app's own `Brand` / `Product` records. This mirrors how an invoice/purchase-bill line item can already be an unlinked, "just for this document" custom row, but skips the product-search step entirely.

Data model. `RateList` (`id`, `title`, `note?`, `createdByUserId`, `createdAt`, `updatedAt`, `deletedAt?`) has many `RateListItem` rows (`onDelete: Cascade`): `serialNo` (display order), `name`, `brand?` (free text, nullable), `unit`, `isNetRate` (default `false`), `discountPercent` (default 0, ignored whenever `isNetRate` is true), `listRate`, and a server-computed `amount`.

Item calculation. Both the client form and the server share one calculation, so they can never drift: for a non-net-rate item, `amount = round((listRate - listRate * discountPercent / 100) * 100) / 100`; for a net-rate item, `amount = listRate` exactly and `discountPercent` is forced to 0. The client-side version lives in `calcRateListItem()` (`src/lib/rateListForm.ts`); the server-side counterpart, `validateAndBuildRateListItems()` in the same file, additionally clamps `discountPercent` into `[0, 100]`, trims/requires `name` and `unit`, requires `listRate >= 0`, and is called identically from both `POST /api/rate-lists` (create) and `PUT /api/rate-lists/[id]` (edit) — the latter bulk-replaces all items in one transaction (`deleteMany` then `create`) rather than diffing, since a rate list carries no stock/ledger side effects that would need reversing/reapplying the way an invoice or purchase bill edit does.

List/detail routes. `GET/POST /api/rate-lists` — list is paginated/searchable via `buildRateListWhere()` / `buildRateListOrderBy()` (`src/lib/rateListQuery.ts`, search matches `title` case-insensitively; sort options `newest` / `oldest` / `title_az` / `title_za`), returning `{data, total}` with item counts and `createdBy`; create takes `title` / `note` / `items` and writes the rate list and its items in one call. `GET/PUT/DELETE /api/rate-lists/[id]` — get returns the full item list ordered by `serialNo`; edit is blocked with a 400 if the rate list is currently in the bin ("restore it before editing"); delete is a plain `deletedAt` soft-delete with no `?permanent=true` path.

PDF. `RateListPrintArea.tsx` (`src/components/rateLists/`) is the one shared PDF/print markup component, mounted two ways: always-mounted on the detail page, and mounted on-demand for the list page's per-row **Preview** button (fetch that row's items, mount the component, wait a frame plus `document.fonts.ready`, generate the blob via `generateInvoicePdfBlob()` against `#rate-list-print-area`, then unmount). The header shows the business logo (`settings.logoUrl`, same option the invoice/purchase-bill PDFs use) in place of a printed date; the table has **no total row** on the PDF by design (kept off the printed document; the on-screen detail page still shows a total for staff). PDF caching (`pdfCache.ts`, entity `"rate-list"`) is keyed on `rateList.updatedAt` + `settings.updatedAt`, so editing either the rate list or the business logo/settings always regenerates instead of serving a stale PDF; the list page's own Preview button never caches at all — it always renders fresh.

Detail page toolbar follows the app-wide detail-page convention (see §9.22): Edit → Regenerate → Preview → Download PDF → Share PDF → Delete, all `size="sm"` buttons. The Share dropdown offers native `navigator.share` / WhatsApp / **Email** — Email calls `/api/send-rate-list` (§9.16) after prompting for a recipient address in a small `Modal`, since there's no linked customer to default one from.

Bulk import. Two entry points feed one shared parser, `parseRateListRows()` / `parsePastedRateListText()` (`src/lib/rateListImport.ts`):

- **"Paste from Excel"** — a `Modal` with a `Textarea`; parses a tab-separated clipboard paste with zero server round-trip via `parsePastedRateListText()`.
- **"Upload .xlsx/.csv"** — posts the file to `POST /api/rate-lists/parse-import` (5 MB cap), which reads `.csv` as plain comma-split text or `.xlsx` via `ExcelJS` (`workbook.xlsx.load()`, `sheet.eachRow()` / `row.eachCell({ includeEmpty: true })`) reducing each row to a `string[]`, then calls the identical `parseRateListRows()`. This route writes nothing to the database — it only returns parsed JSON; the client merges the rows into the form's own items state.

The parser's column detection (`detectColumns()`) matches a header row by column name via an ordered list of regexes checked top-to-bottom, first match per column wins (`COLUMN_PATTERNS`): serial number (`/^(s\.\? \s*no\.\?|sr\.\? \s*no\.\?|#)$ /i`), then `listRate` (`/list\s*rate|^rate$|^price$/i`) is checked **before** the looser `amount` pattern so a "List Rate" header isn't mis-claimed by the amount matcher, then `amount` (`/amount/i`), `name` (`/name|item|chemical|product|description/i`), `brand` (`/brand/i`), `unit` (`/unit/i`), `discount` (`/discount/i`). A row is treated as a real header once at least 2 columns are recognized (`looksLikeHeader`); otherwise the parser falls back to a **positional guess by column count**: 7+ columns → `S.No, Name, Brand, Unit, Discount, List Rate, Amount` (the shape a supplier's own printed rate list commonly uses); 5+ columns → `Name, Brand, Unit, Discount, List Rate` (this app's own item order); 3+ columns → `Name, Unit, List Rate`; otherwise `Name, List Rate`. A discount cell is parsed by `parseDiscountCell()`: an empty cell or one containing "net rate" (case-insensitive) sets `isNetRate: true` and `discountPercent: "0"`; otherwise the % sign is stripped and the number clamped into `[0, 100]`. A row is skipped if it has no `name` or no `listRate` after cleaning (`listRate` is stripped of everything but digits and `.`). A blank `unit` cell defaults to "Nos".

Imported rows are **merged into the form's existing items state**, replacing only the still-empty scaffold rows (so importing into a fresh form doesn't leave a stray blank row behind), rather than saved directly — the user still reviews/edits every imported row before hitting Save, identical to a manually-typed row. Any future rate-list-adjacent bulk-entry feature is expected to extend this one parser rather than write a second one.

Excel export. Both the detail page ("Export Excel" toolbar button) and the list page (a per-row "Export" action, fetching that row's items first) export a rate list's items to `.xlsx` via the app's existing generic `downloadXlsx()` helper (`src/lib/downloadXlsx.ts`) plus `POST /api/export-xlsx` — the same shared infrastructure Credit Notes, Sales Reports, and Purchase Reports already use, so Rate Lists needed no new export endpoint, only a column mapping.

Current gaps (deliberate MVP scope, unresolved as of this edition):

- No public, token-based share link yet — a customer cannot view/download a rate list without logging in.
- No Bin restore/permanent-delete UI — see §9.13.
- The paste-import Modal's submit button (`disabled={!pasteText.trim()}`) still has no `FormField` /inline-error UI wired up, unlike the rest of the app's forms (see §9.20's form-validation convention) — flagged as a pending task, not yet fixed as of this edition.

9.19 Configurable Document Numbering & the Indian Financial Year (new module, 2026-08-09/10)

Purpose: let an admin choose how invoice, purchase-bill, and credit-note numbers are formatted — instead of the fixed `SH-YYYY-0001` scheme hardcoded in the 2nd edition — so a business already using a different manual numbering convention before adopting this app doesn't have to change habits, and so any of the three document types can each independently use a different layout. Configured from **Settings → Document Numbering** (admin-only).

Layouts. `src/lib/documentNumbering.ts` defines `NUMBER_FORMATS: Record<NumberFormatId, NumberFormatDef>` with three entries:

NumberFormatId	Rendered example	Notes
<code>prefix_fy_seq</code>	SH-2026-27-0001	This app's original layout; 4-digit zero-padded sequence, prefix auto-derived from the business name. This is the default when nothing has been explicitly configured yet in Settings → Document Numbering (reverted to this default 2026-08-17 after a mid-development fallback change to <code>seq_fy</code> was found to have never been carried into this app's own documentation — see <code>DEVELOPMENT_LOG.md</code> Eras 9 and 11).
<code>seq_fy</code>	18/2026-27	No prefix, no zero-padding — matches a numbering scheme many businesses already used manually on paper before adopting this app. Fully selectable from Settings, just not the unconfigured default.
<code>prefix_seq_fy</code>	SH-18/2026-27	Keeps a prefix, no zero-padding.

Each `NumberFormatDef` carries a `render(prefix, yearLabel, seq)` function, an `example(prefix)` for the Settings-page live preview, a `matcher(prefix, yearLabel)` regex that extracts the sequence back out of an already-formatted number, and a `dbFilter(prefix, yearLabel)` that narrows Prisma candidate rows (`{startsWith?, endsWith?}`) before the regex scan runs in JS. The `matcher/dbFilter` split exists because the sequence is **not always the number's fixed-width, right-most segment** once the layout is admin-chosen — `seq_fy` puts the sequence *first* and never zero-pads, so a naive `orderBy` string-sort would incorrectly rank "9/2026-27" above "10/2026-27".

Number generation. `computeNextNumber(existingNumbers, formatId, prefix, yearLabel, override):`

1. Resolves the format via `resolveNumberFormat(formatId)` (falls back to `prefix_fy_seq` if `formatId` is null/unrecognized).
2. Calls `findMaxSequence(existingNumbers, format.matcher(prefix, yearLabel))`, which regex-matches every candidate number already narrowed by `numberFormatDbFilter()` and returns the true numeric maximum (not a lexicographic one).
3. If a `nextXNumberOverride` is set on `BusinessSettings` **and** is strictly greater than that max, it wins for exactly this one call (`overrideUsed: true`) — the caller (the route handler, inside the same transaction) is responsible for clearing the override field back to `null` once `overrideUsed` comes back true, so a one-time "jump the counter" admin action never silently re-applies to a later document.
4. Otherwise the sequence is `max + 1`.
5. Returns `{ documentNumber: format.render(prefix, yearLabel, sequentialNumber), overrideUsed }`.

Prefix resolution. `deriveDefaultPrefix(businessName)` auto-suggests an invoice prefix from the business name's initials (e.g. "Science Hub" → "SH", matching what this app always hardcoded before the prefix became configurable) when `BusinessSettings.invoiceNumberPrefix` is null; purchase bills default to the fixed literal "PB", credit notes to "CN", both also independently overridable.

Financial Year. `getIndianFinancialYear(date)` returns the FY start year for any date under the Apr–Mar boundary (`month >= 3 ? year : year - 1` — a January-2027 date belongs to the FY that started April 2026). `formatFinancialYearLabel(startYear)` renders it as the "2026-27" label printed on documents. All three document types generate their FY segment from their own semantically-relevant date — the invoice's `date`, the purchase bill's

`billDate`, the credit note's own `date` — never "now", so a late-entered document for an earlier period numbers correctly.

Admin UX and constraints. Every field (layout, prefix, next-number override) per document type is independently optional, always editable (no one-time lock — existing already-generated documents are never renumbered by a later settings change), confirm-dialog gated with a live rendered-example preview, and logged to `ActivityLog`. Switching layout mid-year restarts *that layout's own* sequence from 1, since a differently-shaped number literally cannot continue a prior sequence numerically. A dismissible one-time banner (`src/components/ui/InfoBanner.tsx`) nudges toward this Settings page on a business's very first invoice/purchase bill if numbering is still untouched.

Downstream effect on editability: because the year segment of an already-generated number is now baked into the printed document, invoice/purchase-bill `date` / `billDate` are editable after creation (to fix a typo) but rejected if the edit would move the date into a different Financial Year than the one the existing number encodes — see §9.1 and §9.7.

9.20 Shared UI Building Blocks Introduced/Changed Since the 2nd Edition

These are not standalone "modules" a user opens directly, but shared components other modules above depend on; documented here because their behavior materially affects several modules at once.

- **UnitCombo** (`src/components/ui/UnitCombo.tsx`) — a typeable text `Input` plus a filtered suggestion dropdown for any "unit" field (e.g. "Nos", "500 GM"), so an unusual free-text unit is never blocked by a constrained `Select`, while the common short units remain one click away. Used by the invoice and purchase-bill "Add Custom Item" quick-add modals (2026-08-11), replacing two near-duplicate hand-rolled combos. **Exception:** the Rate List items table (§9.18) still uses a plain `Input`, not `UnitCombo` — `UnitCombo`'s suggestion dropdown is `position: absolute`, and that table's wrapper is `overflow-x: auto` for horizontal scrolling on narrow screens, which clips the dropdown before it can render below the fold (confirmed broken and reverted same-day, 2026-08-11). `UnitCombo` should not be reapplied inside any scrolling table/container until it's changed to portal its dropdown to `document.body` the way `Select.tsx` already does.
- **Modal fullscreen variant + footer prop** (2026-08-14) — `Modal` (`src/components/dialogs/Modal.tsx`) gained `variant="fullscreen"` (now the app-wide default; the older "center" variant still exists but nothing currently opts into it) and a `footer` prop so Save/Cancel render outside the scrollable body, sized on `height: auto` with a `max-height` cap so a short form doesn't inherit a tall form's fixed height. All 19 app-wide `Modal` call sites were migrated to this, including Settings' six section popups (§9.12). A `<form>` inside the body and its submit button inside `footer` are DOM siblings, not nested — linked via a matching `id` / `form="..."` attribute (`Button`'s `form` prop forwards this).
- **Mobile breakpoint pass + list pagination/refetch UX** (2026-08-14) — the sidebar/topbar mobile-drawer threshold widened from 768px to 1024px (`DashboardShell.tsx`'s `isMobile()` / `check()`); the separate form-field compact-mode breakpoint was standardized to a consistent 768px everywhere (previously an inconsistent 767/768 mix across `Input` / `Select` / `DatePicker` / `PasswordInput` / `PhoneInput` / `Toast`); `Button` gained a `@media (max-width: 640px)` tier shrinking padding/font-size per size class. `Pagination` (`src/components/ui/Pagination.tsx`) gained a `loading` prop (disables Prev/Next mid-fetch) and now blurs the clicked button before scrolling the nearest `.animate-card` ancestor back into view on page change, so a long list's page change doesn't leave the user scrolled past rows that just changed under them. `FloatingSpinner` (`Spinner.tsx`) is a bare, viewport-centered spinner with a translucent scrim for dimming content during a background refetch, distinct from `OverlayLoader`'s solid blocking card used during a save.

None of these change a module's business rules — they are documented here because several of the modules above (Invoices, Purchase Bills, Rate Lists, Settings) render through them.

10. Invoice Flow -- Step by Step

1. User clicks "New Invoice".
2. Selects an existing customer, or fills in a brand-new one inline (name + phone required).
3. Adds line items: product, quantity, price, discount %, GST rate (defaults from the product). A line item can also be an unlinked "custom item" (e.g. "Delivery Charges") with no `productId` — these may repeat freely, unlike real products, which can each appear only once per invoice.
4. Chooses Place of Supply (mandatory) — the app compares it to the business's own state to decide if this is an inter-state sale.
5. Optionally enters a **Transportation Charge** and its own GST rate — a separate, productless freight/service line, distinct from every product line item (see 10.1 below).
6. Client-side validation: quantities > 0, prices ≥ 0, no duplicate products, due date not before the invoice date.
7. A warning (not a block) appears if any item would sell more than is currently in stock.
8. Submit → `POST /api/invoices` — the server re-validates everything independently (never trusts the browser): item quantities/prices/discount range, transport charge/rate non-negative, customer existence, place of supply presence.
9. Server computes GST per line (discount is applied first, then GST on what's left), then decides CGST+SGST or IGST for the whole invoice, and separately computes the transport charge's own GST.
10. Server generates the next sequential invoice number and, in the very same database transaction, decrements stock for every stocked line item.
11. Invoice is saved; the user lands on its detail page.
12. Optionally: record a payment, generate/download/print the PDF, or email it directly to the customer.
13. Later: the invoice can be edited (reverses old stock, re-applies new stock, blocked once fully paid), have its date corrected within the same financial year, have a return/credit note issued against it, or be soft-deleted (which restores its stock and can be undone from the Recycle Bin — invoices are exempt from the 30-day auto-purge, see Chapter 11/CLAUDE.md's Recycle Bin section).

10.1 Transportation charge (new since 2nd Edition)

Update: Invoices (and, symmetrically, purchase bills) now carry an optional **transportation/freight charge** as three new columns on the document: `transportCharge`, `transportChargeGstRate`, `transportChargeGstAmount`. This is not a line item against a product — it is a standalone charge with its own GST rate, added to the grand total but deliberately **kept out of** both the per-rate item tax breakdown and the CGST/SGST/IGST split (which stay pure sums of item-level tax only, computed as `totalGst / 2` each or `totalGst` for IGST — `totalGst` itself never includes the transport charge's GST). The amount and its GST are always recomputed server-side from the raw charge + rate the client sent (`(transportChargeVal * transportChargeGstRateVal) / 100`) — the client-computed figure is never trusted or persisted directly. Both values must be ≥ 0 or the request is rejected with a 400. The grand total becomes:

```
total (before round-off) = subtotal + cgst + sgst + igst + transportCharge + transportChargeGstAmount
```


On the invoice edit route, the transport charge fields use "only touch what was sent" partial-update semantics identical to how `discount` is handled on purchase bills — omitting the field from the PUT body leaves the existing value untouched rather than resetting it to zero.

10.2 Invoice number format is now admin-configurable

Update: the 2nd Edition described the invoice number as a fixed `SH-YYYY-0001` shape. As of the numbering overhaul (`src/lib/documentNumbering.ts`, Settings → Document Numbering), the invoice number's **layout itself** is one of three admin-selectable formats — `prefix_fy_seq` (`SH-2026-27-0001`, the default and this app's own historical shape), `seq_fy` (`18/2026-27`, no prefix, no zero-padding — matching a numbering scheme a business may already have used on paper before adopting the app), or `prefix_seq_fy` (`SH-18/2026-27`). The prefix itself is also independently configurable (auto-derived from the business name via `deriveDefaultPrefix()` — e.g. "Science Hub" → `SH` — unless overridden), and an admin can set a one-time "next number" override that wins for exactly one subsequent invoice if it is higher than what auto-increment would otherwise produce. The **year segment is always a "2026-27" -style Indian financial year label** (1 Apr–31 Mar via `getIndianFinancialYear()` / `formatFinancialYearLabel()`), never the calendar year, regardless of which layout is chosen. Do not assume `SH-YYYY-0001` as a fixed shape anywhere in code or documentation going forward — always resolve it via `computeNextNumber()` / `numberFormatDbFilter()` / `findMaxSequence()` .

Number generation and the invoice `create` both run inside one Serializable transaction (see "Why a transaction matters" below), retried up to 5 times on a genuine write conflict (`P2034`) — this guarantees invoice numbers are never duplicated even under real concurrent use, regardless of which layout is active.

10.3 Invoice date is now editable after creation, guarded by financial year

Update: the invoice's own `date` field — previously implicitly fixed at creation time (defaulting to "now") — is now editable from the invoice edit screen (e.g. to correct a same-week typo). The one hard rule: the date can never be moved across a financial-year boundary, because the invoice number was generated for a specific FY and a date change that crossed into a different FY would desync the printed number from the period it claims to belong to. The edit route computes `getIndianFinancialYear()` for both the existing date and the proposed new date and rejects the request (400) if they differ:

"Invoice date cannot be moved into a different financial year — it would no longer match the invoice number. Delete and re-create the invoice instead if it truly belongs to a different year."

The date is also rejected if it resolves to a future date (`isFutureIstDate()`), and the due date (if present) cannot be earlier than the (possibly newly-edited) invoice date.

10.4 Editable "Bill To" customer, and re-billing to a different customer

Invoice edit additionally allows: (a) fixing a typo in the linked customer's own saved details via a dedicated "Edit" modal on the invoice edit page, which PUTs `/api/customers/[id]` directly and deliberately never touches the invoice's own `placeOfSupply` / `isInterState` even if the customer's state changes (that stays independently editable, since auto-changing it here could silently alter a GST calculation the user didn't intend to touch); and (b) re-pointing the whole invoice at a different customer entirely by sending a new `customerId` in the edit body — the server resolves

and validates the new customer exists (matching the same not-filtered-on- `deletedAt` behavior as create, so a "just for this invoice" one-off customer is still usable).

Why a "transaction" matters here (explained simply)

A database transaction bundles several steps together as one all-or-nothing unit. If two people tried to create an invoice at the exact same split-second, without a transaction they could both be handed the same invoice number, or one person's stock deduction could be silently lost. Science Hub wraps the whole "generate the next invoice number + create the invoice + adjust stock" sequence inside one Serializable transaction (the strictest isolation level a database offers) and automatically retries up to 5 times if the database detects a genuine conflict between two simultaneous attempts — guaranteeing invoice numbers are never duplicated even under real-world concurrent use.

Whether an invoice counts as "inter-state" (and therefore uses IGST instead of CGST+SGST) is independently verified by the server, comparing the invoice's place of supply against the business's own configured state (`src/lib/gstLocation.ts`) — it does not simply trust the browser's flag. The browser still computes and sends the same flag as before (so nothing changes for normal use), but the server's own calculation is authoritative. The one exception: if Business Settings' State field has never been filled in, there is nothing to compare against, so the server falls back to trusting the client-sent flag in that specific case only.

Invoice edits also carry an optimistic-lock guard: the client sends back the `updatedAt` it last saw (`expectedUpdatedAt`), and the server re-checks that condition a second time *inside* the write transaction via an `updateMany` whose `WHERE` clause re-evaluates under the row's own lock — closing a race window where two concurrent edits could otherwise both pass an earlier, separately-run check and the second would silently clobber the first.

11. Stock Flow & the Movement Ledger

Every single change to a product's stock count — for any reason at all — is written as one row in the `StockMovement` table. This table is never edited or deleted in normal use (aside from an admin-only bulk-clear of history, see 11.5); it is a permanent, append-only history. The current stock number on the Product itself is really just a "running total" — the ledger is the detailed proof of how it got there.

11.1 Every reason stock can change

There is **no generic "adjustment" type** in the current schema — every stock change is recorded with one of the following specific `StockMovementType` values (`src/lib/stockMovement.ts`), each mapped to exactly one `documentType` (`invoice` / `purchase_bill` / `credit_note` / `manual`):

Movement type	documentType	When it happens	Direction
<code>sale</code>	<code>invoice</code>	A new invoice is created	Stock decreases
<code>sale_edit_reverse</code>	<code>invoice</code>	An invoice is edited — old quantities put back first	Stock increases (temporarily)
<code>sale_edit_apply</code>	<code>invoice</code>	An invoice is edited — new quantities then deducted	Stock decreases
<code>sale_delete_restore</code>	<code>invoice</code>	An invoice is soft-deleted	Stock increases (restored)
<code>sale_bin_restore</code>	<code>invoice</code>	A deleted invoice is restored from the Recycle Bin	Stock decreases again (the sale re-applies)
<code>purchase</code>	<code>purchase_bill</code>	A new purchase bill is created	Stock increases
<code>purchase_edit_reverse</code>	<code>purchase_bill</code>	A purchase bill's items are edited — old quantities reversed first	Stock decreases (temporarily)
<code>purchase_edit_apply</code>	<code>purchase_bill</code>	A purchase bill's items are edited — new quantities then applied	Stock increases
<code>purchase_cancel</code>	<code>purchase_bill</code>	A purchase bill is cancelled	Stock decreases
<code>purchase_uncancel</code>	<code>purchase_bill</code>	A cancelled purchase bill is un-cancelled	Stock increases
<code>purchase_delete_restore</code>	<code>purchase_bill</code>	A purchase bill is soft-deleted (skipped if already cancelled, to avoid double-reversing)	Stock decreases
<code>purchase_bin_restore</code>	<code>purchase_bill</code>	A deleted purchase bill is restored (skipped if cancelled)	Stock increases
<code>return</code>	<code>credit_note</code>	A credit note is issued against an invoice (goods physically returned)	Stock increases
<code>return_delete_reverse</code>	<code>credit_note</code>	A credit note is soft-deleted	Stock decreases
<code>return_bin_restore</code>	<code>credit_note</code>	A deleted credit note is restored	Stock increases
<code>manual</code>	<code>manual</code>	A manual stock adjustment after a physical count (mandatory reason, not attached to any	Either direction

Movement type	documentType	When it happens	Direction
		sale/purchase/return document)	

This specific, named list of movement types replaced an earlier, single generic "adjustment" catch-all — confirmed still accurate as of this edition: every stock change records precisely why it happened, not just that it happened. Any future stock-affecting feature must extend this union (and its matching entry in `DOCUMENT_TYPE_BY_MOVEMENT_TYPE`) with its own specific type rather than reusing an unrelated one or resurrecting a generic catch-all.

11.2 How a stock change is actually applied

Whenever multiple line items on one document reference the same product (unusual, but possible), the system first adds up all their quantity changes for that product into one single net number (`batchAdjustStock()`'s `deltaByProduct` map), then applies that one combined change in a single, atomic raw-SQL `UPDATE ... FROM (VALUES ...) RETURNING` — this avoids a subtle bug where processing each line item separately could let two changes to the same product overwrite each other instead of adding up, and it does the whole batch in one round trip rather than one query per line item (this matters in production, where the Neon connection is pooled with `connection_limit=1` — every extra round trip serializes instead of overlapping, and a per-item loop is what previously pushed large invoices/purchase bills past the transaction timeout). The very same `UPDATE ... RETURNING` also returns the product's brand-new stock level directly, which is written as that row's `balanceAfter` value — meaning the ledger's "running balance" is always taken directly from the database's own result, never separately re-calculated, so it can never drift out of sync even under heavy concurrent use.

If a raw `UPDATE`'s `WHERE` clause matches fewer rows than there are distinct product IDs in the batch (a stale or already-deleted product reference on a line item), `batchAdjustStock()` throws a `ProductNotFoundError` rather than silently dropping that item's stock adjustment and ledger entry — every route handler that calls it catches this and surfaces a specific 400 telling the user to remove and re-add the affected line item.

11.3 Low stock — a single, unified definition

Confirmed unchanged and still accurate: "is this product low on stock?" is defined in exactly one place, `src/lib/stockStatus.ts` (`isOutOfStock`, `isLowStock`, `needsRestock`) — Out of Stock = stock is exactly 0; Low Stock = stock is above 0 but at or below the minimum-stock threshold; the two are mutually exclusive. Every dashboard, report, and product list/detail screen imports these functions rather than re-deriving the comparison inline. `Product.isLowStock` is additionally a real Postgres `GENERATED ALWAYS AS ("stock" > 0 AND "stock" <= "minStock") STORED` column, mirroring this same logic at the database level so it can be filtered/sorted server-side — application code must never write to it directly.

11.4 Stock & product deletion asymmetry

Confirmed unchanged: the everyday soft-delete check for Products, Brands, and Categories checks both active invoice usage and active purchase-bill usage before allowing a delete; permanent deletion from the Recycle Bin checks *all* purchase bills, including ones already in the bin, which is a stricter check. This is a deliberate two-tier design — a product with only binned purchase-bill history can still be soft-deleted from its everyday page but not yet permanently purged from the bin — not a bug.

11.5 Clearing ledger history

`DELETE /api/stock-movements?type=stock-ledger` (admin-only) bulk-clears the `StockMovement` history rows. This is the one exception to "never edited or deleted" above — it is an explicit, admin-triggered housekeeping action, and it does not change any product's current `stock` value; only the historical audit trail is removed.

12. GST Logic

12.1 The core formula, per line item

```
gross           = quantity × unit price
discount amount = gross × (discount % ÷ 100)
taxable amount  = gross - discount amount      GST is calculated AFTER discount
GST amount      = taxable amount × (GST rate % ÷ 100)
line total      = taxable amount + GST amount
```

This exact formula (`lineBreakdown()` in `src/lib/invoiceCalc.ts` for invoices, `purchaseBillLineBreakdown()` in `src/lib/purchaseBillForm.ts` for purchase bills) is used identically for invoices, purchase bills, and returns/credit notes — confirmed as the same shared logic in all three, not three separately-written (and potentially inconsistent) copies.

12.2 CGST + SGST, or IGST?

India's GST law splits tax two different ways depending on whether a sale crosses a state border:

- **Intra-state sale** (buyer and seller in the same state): the tax is split evenly into CGST (Central GST) and SGST (State GST) — each exactly half the total GST.
- **Inter-state sale** (buyer and seller in different states): the entire tax amount is charged as one line, IGST (Integrated GST) — nothing is split.

In Science Hub, this decision is driven by a single yes/no flag on the invoice ("is this inter-state?"). The invoice-creation screen sets it automatically by comparing the chosen Place of Supply to the business's own registered state, and the server independently re-derives and verifies the same comparison via `deriveIsInterState(placeOfSupply, businessState)` (`src/lib/gstLocation.ts`) rather than simply trusting what the browser sent — confirmed still accurate, applied on both invoice create (`POST /api/invoices`) and invoice edit (`PUT /api/invoices/[id]`). `deriveIsInterState()` returns `null` (rather than a guessed boolean) when either the supplied place-of-supply or the business's own configured state is blank, since there is nothing to compare in that case; both invoice routes fall back to the client-sent flag only in that specific null case.

Purchase bills use the same function symmetrically: a purchase's GST type is derived from the *vendor's* own registered state compared against the business's state (`deriveIsInterState(vendorState, biz.state)`), not something the preparer manually picks — applied on both purchase-bill create and edit (re-derived on every edit in case the vendor was switched or the vendor's own state was corrected after the bill was created).

12.3 Discount

Discounts are entered as a percentage per line item (not a flat rupee amount) and — as shown above — are always subtracted before GST is calculated, exactly matching how Indian tax law expects discounts to be treated for GST purposes. This applies identically to invoices, purchase bills, and returns.

12.4 Transportation/freight charge — kept out of the item tax split

New in this edition: invoices and purchase bills can each carry an optional transportation charge line (`transportCharge` , `transportChargeGstRate` , `transportChargeGstAmount` — see Chapter 10.1). This charge and its GST are deliberately **excluded** from the per-item `taxBreakdown` /CGST-SGST-IGST split, which remains a pure sum of item-level tax only. The transport charge's own GST is computed as a flat $(\text{transportCharge} \times \text{transportChargeGstRate}) / 100$ and added straight into the grand total alongside (not blended into) the item tax:

```
grand total (pre-round-off) = itemsSubtotal + itemsCGST + itemsSGST + itemsIGST
                             + transportCharge + transportChargeGstAmount
```

This keeps the CGST/SGST/IGST figures on the printed invoice/bill representing item-level tax only, which matters for GST filing accuracy — a transport charge is legally its own taxable supply and does not need to inherit the item lines' inter-state/intra-state tax split logic, since in this app's implementation it is charged at its own flat rate regardless (there is currently no separate inter-state/intra-state CGST-SGST-vs-IGST split applied to the transport charge amount itself — it is a single GST-rate line, always). Like every other tax figure in the app, the amount is always recomputed server-side from the client-sent charge and rate, never persisted from a client-computed total.

12.5 Reverse Charge

"Reverse Charge" is a GST-law concept where, in certain transactions, the buyer rather than the seller is responsible for paying the GST to the government. Science Hub lets an invoice be flagged as reverse-charge (a simple yes/no checkbox that prints on the invoice as legally required), but this flag is currently for disclosure only — it does not itself change any of the tax arithmetic on the invoice. Confirmed unchanged in this edition.

12.6 Rounding

After GST (and any transportation charge/GST) is added to the subtotal, the grand total is rounded to the nearest whole rupee via `computeRoundOff()` (`src/lib/roundOff.ts`) — standard commercial rounding, 50 paise and above rounds up — and the small difference this creates (a few paise, positive or negative) is shown as its own explicit "Round Off" line on the invoice/bill, rather than being silently absorbed anywhere. This is identical logic shared by invoice create, invoice edit, purchase-bill create, and purchase-bill edit.

12.7 Worked example

Item	Qty	Price	Discount	Taxable	GST @18%	Line Total
Beaker 500ml (Borosil)	10	120	5%	1,140	205.20	1,345.20

If sold within the same state: CGST 102.60 + SGST 102.60 = 205.20 total GST. If sold to another state: IGST 205.20 in one line. Either way the customer pays the same 1,345.20 before round-off (before any additional transportation charge, if one is added to the invoice).

12.8 Credit notes / returns and GST

When a customer returns goods, the credit note re-uses the exact GST rate that was actually charged on the original invoice line for that product (not whatever the product's GST rate might have since changed to) — a small but important correctness detail, since GST rates can legally change over time and a refund must match what was

originally charged. Confirmed unchanged in this edition. The credit note's own document number follows the same configurable-layout/prefix/FY system as invoices and purchase bills (`BusinessSettings.creditNoteNumberFormat / creditNoteNumberPrefix` , default prefix `CN`) — see Chapter 10.2 for the general numbering mechanism, which applies identically to all three document types.

12.9 Server-side numbering as a GST compliance control

The financial-year-aware, admin-configurable numbering system (`src/lib/documentNumbering.ts` , Chapter 10.2) is itself a GST-adjacent control, not just a cosmetic formatting choice: Indian GST filing periods run on the same 1 April–31 March boundary, and a numbering sequence that silently continued across that boundary (or developed a gap) would complicate reconciling a filing period's invoice/bill/credit-note sequence against what was actually issued. This is also the reasoning behind invoices, purchase bills, and credit notes being exempt from the Recycle Bin's 30-day auto-purge (see Chapter 11 and CLAUDE.md's Recycle Bin section) — permanently deleting a numbered document is only ever an explicit, admin-only, warned action from the Bin page itself, never an automatic timer.

13. API Documentation (Full Reference)

Every endpoint below was confirmed to exist in the current codebase (cross-checked against `CLAUDE.md`'s "API Routes (full list)" table and the actual route handler source). "Auth" shows the minimum permission guard applied — `Session` means `requireSession()` (any logged-in user), `Write` means `requireWriteAccess()` (blocks the read-only `manager` role), `Admin` means `requireAdmin()`, and `Section: x` means `requireSectionAccess(x)` (admin always bypasses). All of `/api/**` additionally sits behind `middleware.ts`'s default-deny baseline — a request with no valid session is rejected before it reaches the handler at all, except for an explicit public allowlist (`/api/auth/*`, `/api/setup`, `/api/settings/branding`).

Every paginated list route below returns `{ data, total }`, not a bare array, and has a **companion `/stats` route** that reuses the exact same `buildXWhere()` filter as its list route to aggregate totals over *every* matching row — not just the current page. This pairing exists for: invoices, products, brands, categories, vendors, purchase bills, purchase-bill payments, sales payments, and credit notes.

13.1 Sales

Method / Path	Auth	Purpose
GET/POST <code>/api/invoices</code>	Session / Write	List invoices — paginated/searchable/sortable (<code>?status</code> , <code>?customerId</code> , <code>?search</code> , <code>?sort</code> , <code>?month</code> , <code>?year</code> , <code>?page</code> , <code>?pageSize</code>), returns <code>{data, total}</code> / Create an invoice — auto-numbers per the configurable layout (default <code>SH-YYYY-0001</code>), decrements stock, computes transport charge GST server-side
GET <code>/api/invoices/stats</code>	Session	Summary totals (outstanding/paid/etc.) over ALL invoices matching the same filters as the list route, independent of its current page
GET/PUT/DELETE <code>/api/invoices/[id]</code>	Session / Write	View → Full edit (reverses+reapplies stock, re-validates returned-qty floor, blocked once fully paid, date edit blocked across a financial-year boundary) → Soft-delete (restores stock, double-delete safe)
POST <code>/api/invoices/[id]/payment</code>	Write	Record a payment, recompute <code>paidAmount</code> /status
PUT <code>/api/invoices/[id]/payment/[paymentId]</code>	Write	Edit an existing payment, recompute invoice status (no DELETE exists for a single payment)
GET/POST <code>/api/invoices/[id]/returns</code>	Session / Write	List / create a credit note against this invoice — validated against paid amount and remaining returnable quantity inside a Serializable transaction; restores stock
DELETE <code>/api/invoices/[id]/returns/[returnId]</code>	Write	Soft-delete a credit note (reverses its stock effect)
GET/POST <code>/api/customers</code>	Session / Write	List customers (with <code>createdBy</code>) / Create customer
GET/PUT/DELETE <code>/api/customers/[id]</code>	Session / Write	View → Edit (blocked if in bin) → Soft-delete (blocked if active invoices exist)

Method / Path	Auth	Purpose
GET <code>/api/payments</code>	Session	All sales payments — paginated (<code>paymentQuery.ts</code>), returns <code>{data, total}</code>
GET <code>/api/payments/stats</code>	Session	Total/count summary over ALL matching sales payments, independent of the list route's current page
GET <code>/api/credit-notes</code>	Session	All credit notes (non-deleted <code>Return</code> rows) across every invoice — paginated/searchable/sortable (<code>?search</code> , <code>?sort</code> , <code>?month</code> , <code>?year</code> , <code>?page</code> , <code>?pageSize</code> , via <code>buildReturnWhere</code> / <code>buildReturnOrderBy</code> in <code>creditNoteQuery.ts</code>), returns <code>{data, total}</code> ; each row is annotated with a best-effort <code>createdBy</code> name matched from the linked invoice's <code>create_return</code> activity-log entry (a <code>Return</code> has no direct creator foreign key of its own)
GET <code>/api/credit-notes/stats</code>	Session	Summary totals over ALL matching credit notes, independent of the list route's current page — all-time total count/credited amount, plus a period (month/year-filtered, or current calendar month by default) count/credited amount and the list of years that have any credit notes
GET/POST <code>/api/rate-lists</code>	Session / Write	List rate lists — paginated/searchable/sortable (<code>?search</code> , <code>?sort</code> , <code>?page</code> , <code>?pageSize</code> , via <code>rateListQuery.ts</code>), returns <code>{data, total}</code> (item counts, <code>createdBy</code>) / Create a rate list with all its items in one call — server-validates the title and every item (<code>validateAndBuildRateListItems()</code>), computing each item's amount server-side (<code>isNetRate ? listRate : listRate - listRate×discountPercent/100</code> , rounded to 2dp) rather than trusting a client-sent amount
GET/PUT/DELETE <code>/api/rate-lists/[id]</code>	Session / Write	Get (with items, ordered by <code>serialNo</code>) → Edit (bulk-replaces all items — delete-and-recreate inside one transaction, since a rate list has no stock/ledger side effects to reverse; blocked if the rate list is in the bin) → Soft-delete (<code>deletedAt</code> ; no bin-restore UI exists yet for this entity — see Pending Tasks in CLAUDE.md)
POST <code>/api/rate-lists/parse-import</code>	Write	Parses an uploaded <code>.xlsx</code> (via ExcelJS) or <code>.csv</code> (plain comma-split) file into rate-list item rows via the shared <code>parseRateListRows()</code> (<code>src/lib/rateListImport.ts</code>); capped at 5MB; writes nothing to the database — returns <code>{items, skipped}</code> JSON only, and the New/Edit page merges the parsed rows into its own in-memory items state so they remain reviewable/editable before Save, exactly like a manually-typed row
POST <code>/api/send-rate-list</code>	Write, rate-limited (20/15min per user)	Emails a rate list PDF to an address typed in at share time (a rate list has no linked customer/recipient to default one from, unlike <code>/api/send-invoice</code>); Gmail credentials come from <code>BusinessSettings</code> with an environment-variable fallback; validates recipient email format and caps the attached PDF at 10MB

13.2 Purchases

Method / Path	Auth	Purpose
GET/POST <code>/api/vendors</code>	Session / Write	List — paginated/searchable (<code>vendorQuery.ts</code>), returns <code>{data, total}</code> (active bill counts) / Create vendor (requires phone + address)
GET/PUT/DELETE <code>/api/vendors/[id]</code>	Session / Write	Detail (purchase bills) → Edit → Soft-delete (blocked if active bills exist)
GET/POST <code>/api/purchase-bills</code>	Session / Write	List — paginated/searchable/sortable (<code>?status</code> incl. synthetic <code>overdue</code> , <code>?vendorId</code> , <code>?search</code> , <code>?sort</code> — <code>newest/oldest/vendor_az/vendor_za/amount_high/amount_low/balance_high</code> , <code>?month</code> , <code>?year</code> , <code>?page</code> , <code>?pageSize</code> , via <code>purchaseBillQuery.ts</code>), returns <code>{data, total}</code> / Create — auto-numbers per the configurable layout (default <code>PB-YYYY-0001</code>), server recomputes GST/totals/transport-charge GST from scratch, increments stock, optional inline payment
GET <code>/api/purchase-bills/stats</code>	Session	Summary totals (total/paid/pending, overdue count, available years) over ALL matching bills, independent of the list route's current page
GET/PUT/DELETE <code>/api/purchase-bills/[id]</code>	Session / Write	Get → Edit (reverses+reapplies stock on an item change; handles cancel/un-cancel with its own stock reversal/reapplication; blocks item/discount edits on paid or cancelled bills; date edit blocked across a financial-year boundary, same as invoices) → Soft-delete (reverses stock unless already cancelled, double-delete safe)
POST <code>/api/purchase-bills/[id]/payment</code>	Write	Record a payment against a bill, recompute status
GET <code>/api/purchase-bills/payments</code>	Session	All purchase payments — paginated (<code>purchasePaymentQuery.ts</code>), returns <code>{data, total}</code>
GET <code>/api/purchase-bills/payments/stats</code>	Session	Total/count summary over ALL matching purchase payments, independent of the list route's current page
POST/DELETE <code>/api/purchase-bills/upload</code>	Write	Upload a bill attachment to Vercel Blob (size/MIME/magic-byte validated) / Delete an orphaned, never-attached upload
GET <code>/api/purchase-reports</code>	Section: <code>reports_purchases</code>	<code>?type=summary outstanding category stock-ledger</code> — <code>summary</code> / <code>category</code> exclude cancelled bills from spend totals; <code>stock-ledger</code> returns the entire stock movement ledger (not purchase-only, despite the route name)

13.3 Catalogue

Method / Path	Auth	Purpose
GET/POST <code>/api/products</code>	Session / Write	List — paginated/searchable/sortable (<code>?search</code> , <code>?stockFilter</code> , <code>?sort</code> , <code>?page</code> , <code>?pageSize</code> , via <code>getProducts()</code> in <code>db.ts</code>), returns <code>{data, total}</code> , each row with <code>createdBy</code> / Create product
GET <code>/api/products/stats</code>	Session	Stock/value summary over ALL matching products, independent of the list route's current page

Method / Path	Auth	Purpose
GET/PUT/DELETE <code>/api/products/[id]</code>	Session / Write	View (incl. last 15 stock movements) → Edit → Soft-delete (blocked if used in invoice or active purchase-bill line items)
POST <code>/api/products/[id]/adjust-stock</code>	Write	Manual stock correction after a physical count — requires a written reason, writes a "manual" ledger row
GET/POST <code>/api/brands</code>	Session / Write	List — paginated/searchable (<code>brandQuery.ts</code>), returns <code>{data, total}</code> (product counts, <code>createdBy</code>) / Create brand
GET/DELETE <code>/api/brands/[id]</code>	Session / Write	Detail (assigned products) / Soft-delete (blocked if products assigned or used in invoices/active purchase bills)
GET/POST <code>/api/categories</code>	Session / Write	List — paginated/searchable (<code>categoryQuery.ts</code>), returns <code>{data, total}</code> (product counts) / Create category
GET/PUT/DELETE <code>/api/categories/[id]</code>	Session / Write	Detail / Rename / Soft-delete (same blocking rules as brands)

Update: the 2nd Edition listed `GET/POST /api/brands` and `GET/POST /api/categories` as both requiring `Write` for the GET verb — corrected above to `Session / Write` (GET only needs a valid session; only POST/PUT/DELETE require write access), matching every other list route's convention and the actual `requireSession()` / `requireWriteAccess()` calls in each handler.

13.4 Reports & Filing

Method / Path	Auth	Purpose
GET <code>/api/reports</code>	Session (some sub-types Section-gated on the page level)	<code>?type=summary outstanding stock sales-dashboard purchase-dashboard combined-dashboard gst-summary</code> — <code>purchase-dashboard</code> 's Top Vendors and monthly spend/paid totals exclude cancelled bills
GET <code>/api/gst-filing</code>	Admin, or <code>reports_sales</code> + <code>reports_purchases</code> both	Builds the GSTR-style filing package (Sales/Purchase Register, B2B/B2C split, Credit Notes, HSN Summary, net-GST-payable summary) for a chosen month or financial year — JSON, or <code>?format=zip</code>
POST <code>/api/export-xlsx</code>	Session	Generic "rows → <code>.xlsx</code> file" export, shared by Credit Notes / Sales Reports / Purchase Reports / Rate Lists (capped at 20,000 rows / 50 columns)

13.5 System & Admin

Method / Path	Auth	Purpose
GET <code>/api/search</code>	Session	Global search (<code>?q=</code>) across 7 entity types — invoices, customers, products, vendors, purchase bills, brands, categories; 5 results per group
GET <code>/api/bin</code>	Write	Auto-purges most bin item types older than 30 days (invoices, purchase bills, and credit notes are exempt — retained indefinitely),

Method / Path	Auth	Purpose
		then lists everything still in the bin with <code>daysLeft</code> (<code>-1</code> for the exempt types) / <code>deletedBy</code> / <code>protectedReason</code>
POST/DELETE <code>/api/bin/[type]/[id]</code>	Write / Admin	Restore an item (re-applies stock, double-restore safe) / Permanently delete one item (admin-only, per-type FK checks, blob cleanup) — the only way to hard-delete an invoice/purchase bill/credit note
DELETE <code>/api/bin/empty</code>	Admin	Permanently purge the entire bin at once, in FK-safe order
GET/PUT <code>/api/settings</code>	Session / Admin	View (non-admins don't see Gmail fields) → Update business settings, incl. bank details and Gmail send-from (both encrypted at rest) and document-numbering configuration
GET <code>/api/settings/branding</code>	Public — no login required	Business name/tagline/logo, needed on the unauthenticated login page itself
POST/DELETE <code>/api/settings/logo</code>	Admin	Upload / remove the business logo (Vercel Blob, size/MIME/magic-byte validated)
GET <code>/api/settings/ifsc-lookup/[code]</code>	Admin	Server-side proxy to Razorpay's public IFSC directory, to look up a bank branch from its IFSC code
GET/POST <code>/api/admin/users</code>	Admin	List users (with invoice counts) / Create staff/admin/manager accounts
GET/PUT/DELETE <code>/api/admin/users/[id]</code>	Admin (self-edit allowed for own profile)	Manage one user; blocks deleting yourself or the last remaining admin
GET/PUT <code>/api/admin/profile</code>	Session	View/update your own profile and password; includes a <code>resolveSessionUser</code> fallback for old JWTs missing an <code>id</code> claim
GET <code>/api/admin/activity</code>	Admin	Browse the activity log (<code>?userId</code> , <code>?limit</code> max 500, <code>?offset</code>)
DELETE <code>/api/admin/activity/[id]</code>	Admin	Delete one log entry
GET/POST <code>/api/admin/permissions</code>	Admin	List / grant the six <code>ProtectedSection</code> grants for non-admin users
DELETE <code>/api/stock-movements?type=stock-ledger</code>	Admin	Bulk-clear the stock ledger's history rows (does not change current stock levels)
POST <code>/api/send-invoice</code>	Write, rate-limited (20/15min)	Email an invoice PDF via Gmail; credentials from <code>BusinessSettings</code> with environment-variable fallback; returns 503 with a clear message if not configured
POST <code>/api/send-rate-list</code>	Write, rate-limited (20/15min)	See 13.1 above — same pattern as <code>/api/send-invoice</code> , but the recipient address is always supplied at share-time since a rate list has no linked customer
POST <code>/api/setup</code>	None — disabled once any user exists, or in production	Create the very first admin account (and, in the seed script, demo data)

Method / Path	Auth	Purpose
POST <code>/api/auth/find-email</code>	Rate-limited, no login	Search users by name, return a masked email
POST <code>/api/auth/forgot-password</code>	Rate-limited, no login	Generate a 1-hour password-reset token and send it by email; always responds <code>{ok:true}</code> regardless of whether the email matched a user (anti-enumeration)
POST <code>/api/auth/reset-password</code>	Rate-limited, no login	Validate the reset token, set the new password, mark the token used (single-use)
* <code>/api/auth/[...nextauth]</code>	—	NextAuth's own internal login/session handler

Update: every list-route/stats-route pairing above (`invoices` , `products` , `brands` , `categories` , `vendors` , `purchase-bills` , `purchase-bills/payments` , `payments` , `credit-notes`) is now a *confirmed, current* pattern shared by nine endpoints, up from the pagination refactor's original set at the 2nd Edition's time of writing — `rate-lists` also follows the same paginated-list shape (`{data, total}` , `?search` / `?sort` / `?page` / `?pageSize`) but does **not** yet have its own `/stats` companion route, since the Rate Lists list page has no summary stat cards to back.

14. Frontend Architecture

14.1 Pages & layouts

Every authenticated page lives under the `(dashboard)` route group and is wrapped by one shared component, `DashboardShell`, which provides the sidebar, topbar, global search box, and the client-side authentication gate (redirecting to `/login` the moment a session becomes invalid). Public pages (login, forgot/reset password, find-email) sit outside this shell entirely.

14.2 State management — deliberately simple

There is no heavyweight global state library (no Redux, no Zustand). Instead, server data is managed by one small, custom hook, `useFetch(url)`, backed by a single shared in-memory cache (`src/lib/useCache.ts`): the first component to ask for a URL fetches it and stores the result; every other component asking for that same URL, anywhere in the app, instantly gets the same cached value and is notified the moment it changes. After any create/edit/delete, the calling page explicitly tells the cache to refresh (`mutate()`) or discards the stale entry (`bustCache()` / `bustCachePrefix()`) — there is no automatic timed expiry; a cached value lives until something explicitly invalidates it. `useFetch` also throws (via its internal `parseOrThrow`) on a non-2xx response instead of quietly returning the error body as if it were data, so a route returning `{error: "..."} with a 401/404/500 status can't be mistaken for a successful payload downstream.`

14.3 Forms & validation

Every input field's validation rule (required, email format, GSTIN format, PAN format, IFSC format, phone number, positive number, etc.) is defined once in `src/lib/validation.ts` and reused on both sides: the browser checks it immediately for a responsive experience, and the very same rule set is re-run on the server before anything is saved — so the server is never simply trusting what the browser already checked.

Every form in the app is built from the same shared primitives (`Input`, `Select`, `Textarea`, `FormField`, `PasswordInput`) plus `rules` / `validate` / `validateForm` — hand-rolled `<input>` /ad-hoc validation is not permitted outside a small allowlist of native-only types (`search`, `color`, `file`, `checkbox`, `radio`). Two distinct disable-button conventions apply depending on the form's purpose: an **edit** form (fields start pre-filled) keeps an explicit "every mandatory field non-empty" check on its submit button in addition to `saving`, so an emptied required field can't slip through; a **new/create** form (fields start empty) gates its submit button on `saving` alone and relies on the existing `validate()` / `validateForm()` call to populate per-field `FormField` errors — a disabled-by-default button on a still-empty form gives no signal about which field is missing.

14.4 The app-wide `Modal` component

Update: as of 2026-08-14, every popup dialog in the app — 19+ call sites and counting (Settings' six section editors, both invoice/purchase-bill "Add Custom Item" quick-add modals, the inline vendor/customer create modals, Categories/Brands Add & Rename, Admin's New/Edit User, both Rate List modals, and more) — is built on one shared component, `Modal` (`src/components/dialogs/Modal.tsx`), rather than each page rolling its own centered-box markup. Reading the component directly:

- It is portaled to `document.body` via `createPortal`, so it always covers the full viewport regardless of where in the DOM it's mounted — nesting it under an ancestor that has a CSS animation applied (e.g. a page section using `animate-card`) would otherwise trap it inside that ancestor's own stacking context and let later page content paint over it.
- `variant` is `"center"` | `"fullscreen"`, defaulting to `"center"` in the component's own type signature, but **every current call site in the app passes `"fullscreen"`** — the plain centered variant still exists in code but nothing currently opts into it. `"fullscreen"` covers the entire viewport on mobile (no small centered card to scroll a tall form inside of) and becomes a large, content-sized centered panel — capped by a `max-height`, not a fixed height — on wider screens, so a short form (e.g. Terms & Conditions) doesn't inherit a tall form's (e.g. Document Numbering) fixed height.
- An optional `footer` prop renders outside the scrollable `body` region, pinned to the bottom, so a form's Save/Cancel actions stay reachable without scrolling to the end of a long fullscreen form.
- Because `footer` and `children` (the scrollable `body`) are DOM siblings, not parent/child, a `<form>` placed in `children` cannot simply nest its submit button inside `footer`. The convention is to give the `<form>` an `id` and have the footer's submit `<Button>` reference it via a `form="..."` prop (the shared `Button` component forwards this attribute straight through to the underlying `<button>`), rather than nesting the button inside the form.
- Escape-to-close and backdrop-click-to-close are both wired (`onKey` 's `Escape` handler and the `.backdrop` div's `onClick`), and are deliberately kept as two separate guarded paths rather than one combined "any dismiss gesture" handler, so a future change to one doesn't silently disable the other. Focus is trapped inside the open dialog (`Tab` / `Shift+Tab` wrap between the first and last focusable element) and returns to whatever triggered the modal on close.

Detail-page action toolbars follow one shared convention across every entity (invoice, purchase bill, rate list, ...), established on `sales/invoices/[id]/page.tsx`: a `page-header` with `<Breadcrumb>` + a small meta line on the left, and every action rendered as `<Button size="sm">` (never `"md"` / `"full"` in a toolbar) on the right, in a fixed order — entity-specific mutations first, then **Regenerate** (discards a cached PDF variant), **Preview**, **Download PDF**, **Share PDF** (a dropdown: native share/WhatsApp/Email-or-Copy-Download), then **Delete** last.

14.5 Shared UI components

Component	Purpose
<code>Button</code>	One consistent button across the whole app — primary/secondary/danger styles, loading spinner built in; gained a <code>@media (max-width: 640px)</code> tier (2026-08-14) that shrinks padding/font-size per size class (<code>sm</code> / <code>md</code> / <code>lg</code> / <code>full</code>) so toolbar buttons don't overflow on phone-width screens
<code>Input</code> / <code>Select</code> / <code>Textarea</code> / <code>FormField</code>	Consistent form fields with label, required-marker, and inline error message
<code>UnitCombo</code>	Typeable free-text input + a filtered suggestion dropdown for "unit" fields (e.g. "Nos", "500 GM") — any value can still be typed freely, the dropdown is just a shortcut. Shared by the invoice/purchase-bill "Add Custom Item" quick-add modals. Not used inside the Rate List items table — see 14.6
<code>Badge</code> / <code>StatusBadge</code>	Colour-coded pill labels — e.g. green "Paid", amber "Partial", red "Unpaid"
<code>Toast</code>	Temporary pop-up notifications for success/error/warning messages
<code>ConfirmDialog</code>	"Are you sure?" modal used before every destructive action
<code>Modal</code>	The single popup-dialog component for everything else (see 14.4)

Component	Purpose
PdfCopyDialog	Lets the user choose "Original" and/or "Duplicate" copy before generating an invoice PDF
Pagination	Standard page-by-page browsing for long lists (see 14.7)
Skeleton	Grey placeholder blocks shown while data is loading, so the page never flashes empty
Spinner / FloatingSpinner	FloatingSpinner (2026-08-14) is a bare spinner fixed to the viewport center with a translucent scrim and no card, for dimming still-visible content during a background refetch — distinct from OverlayLoader , which is a solid card meant for blocking a whole page/modal during a save
GlobalSearch	The topbar search box, debounced (waits 250ms after typing stops) and cancels any still-in-flight previous search

14.6 Rate List forms

`sales/rate-lists` is a standalone, downloadable price-sheet/catalog builder, deliberately **not** linked to the `Product` catalog — a rate list's brand/unit strings (e.g. "QUALIGENS", "500 GM") routinely don't match this app's own `Brand / Product` records. `RateListFormBody.tsx` is the shared two-column layout used by both the New and Edit Rate List pages: a left column with the Details card (Title/Note) and the items table, and a right column with a Summary sidebar (item count + whatever validation/action footer the page passes in) — mirroring `PurchaseBillFormBody` 's structure so the two forms can't drift apart.

The items table (`RateListItemsTable.tsx`) uses a plain `Input` for its Unit column instead of `UnitCombo`. This is a deliberate, documented exception: `UnitCombo` 's suggestion dropdown is `position: absolute`, and the table's own scroll wrapper is `overflow-x: auto` (for horizontal scrolling on narrow screens), which clips the dropdown before it can render below the fold. This was tried and reverted the same day it was found broken (2026-08-11) — `UnitCombo` should not be reapplied inside any scrolling table/container until it's changed to portal its dropdown to `document.body` the way `Select.tsx` already does.

The items table also has a bulk-import entry point — "Paste from Excel" (client-side, tab-separated clipboard parsing) and "Upload .xlsx/.csv" (server-side via `POST /api/rate-lists/parse-import`) — both funneling through one shared parser, `parseRateListRows()` / `parsePastedRateListText()` in `src/lib/rateListImport.ts`, so a future bulk-entry feature has one parser to extend rather than a second one to write.

14.7 Pagination and mobile breakpoints

Update (2026-08-14): `Pagination` (`src/components/ui/Pagination.tsx`) gained a `loading` prop that disables its Prev/Next buttons while a page's data is still in flight, and its click handlers now do two things before calling `onPage()`: blur the clicked button, then call `scrollIntoView({behavior: "smooth", block: "start"})` on the nearest `.animate-card` ancestor. The blur step is required first — a still-focused element that becomes `disabled` on the next render has its focus force-cleared by the browser, which fires its own scroll adjustment that fights and cancels the smooth one. Without this, changing page on a long list left the user scrolled down past table rows that had just changed under them, with no visual cue anything happened. The same scroll-to-top-on-page-change idea is hand-rolled in `admin/page.tsx` 's activity-log pagination (which pre-dates the shared component gaining this behavior) — treat it as the app's general pagination convention, not something unique to one component.

Separately, a broader mobile-responsiveness pass (also 2026-08-14) touched three independent breakpoints that must not be conflated:

- **1024px** — `DashboardShell.tsx`'s `isMobile() / check()` mobile-drawer threshold (`window.innerWidth < 1024`), widened from 768px so tablet-width screens get the sidebar overlay drawer too.
`DashboardShell.module.css` and `GlobalSearch.module.css` each carry their own `@media (max-width: 1024px)` rule that must stay numerically in sync with this JS threshold, or the sidebar/search UI and the JS-driven layout state disagree at tablet widths.
- **768px** — the plain form-field compact-mode breakpoint, standardized across `Input / Select / DatePicker / PasswordInput / PhoneInput / Toast` (previously an inconsistent 767/768 mix across those files).
- **640px** — `Button`'s new shrink tier (see 14.5), unrelated to either of the above.

14.8 Theming

Light/dark mode and a customisable accent colour are both controlled purely through CSS variables and a small script that runs before the page even paints, preventing a "flash of the wrong theme" on load (`src/app/layout.tsx`'s inline script + `suppressHydrationWarning`).

15. Backend Architecture

There is no separate "controller / service / repository" layering in this codebase — it is intentionally kept simple. Almost every API route file directly: (1) checks permissions, (2) validates input, (3) talks to Prisma, (4) logs the action, (5) tells the cache to refresh. A small number of shared helper modules exist for logic that would otherwise be duplicated:

Helper module	Responsibility
<code>apiAuth.ts</code>	The four permission-guard functions: <code>requireSession()</code> , <code>requireAdmin()</code> , <code>requireWriteAccess()</code> (blocks the read-only <code>manager</code> role), <code>requireSectionAccess(section)</code>
<code>invoiceCalc.ts</code> , <code>purchaseBillForm.ts</code> , <code>roundOff.ts</code> , <code>ratelistForm.ts</code>	Shared GST/discount/rounding math, so invoices, purchase bills, and rate list items can never compute tax/amounts two different ways
<code>stockMovement.ts</code>	The single place every stock change must go through — <code>batchAdjustStock()</code> inside the same transaction, one of a fixed set of specific <code>StockMovementType</code> values (no generic "adjustment" type)
<code>invoiceReturns.ts</code>	Prevents editing an invoice's quantities below what's already been returned against it
<code>blobStorage.ts</code>	File-upload URL allowlisting (<code>isPurchaseBillBlobUrl()</code> / <code>isLogoBlobUrl()</code>) and best-effort cleanup for attachments/logos
<code>activity.ts</code>	Writes to the audit log; deliberately can never throw an error that would break the real operation
<code>ratelimit.ts</code>	Basic abuse-prevention counters for login, password reset, and email sending — in-memory, see Chapter 19.6
<code>crypto.ts</code>	Encrypts/decrypts the sensitive settings fields (Gmail app password, bank account number)
<code>gstFiling.ts</code> / <code>gstFilingZip.ts</code>	Builds the GST filing report and its downloadable ZIP (and, since 2026-08-12, an <code>.xlsx</code> export alongside JSON/ZIP)
<code>documentNumbering.ts</code>	Shared invoice/purchase-bill/credit-note numbering: Indian financial year derivation, the admin-configurable <code>NUMBER_FORMATS</code> layout registry, and <code>computeNextNumber()</code> / <code>numberFormatDbFilter()</code> / <code>findMaxSequence()</code>
<code>stockStatus.ts</code>	The single shared definition of "out of stock" vs. "low stock" (<code>isOutOfStock</code> / <code>isLowStock</code> / <code>needsRestock</code>)
<code>gstLocation.ts</code>	<code>deriveIsInterState()</code> — server-side inter-state/intra-state derivation, never trusting the client's flag
<code>pdfCache.ts</code>	Client-side IndexedDB cache for generated PDF blobs, keyed by entity + a variant key (see Chapter 17)

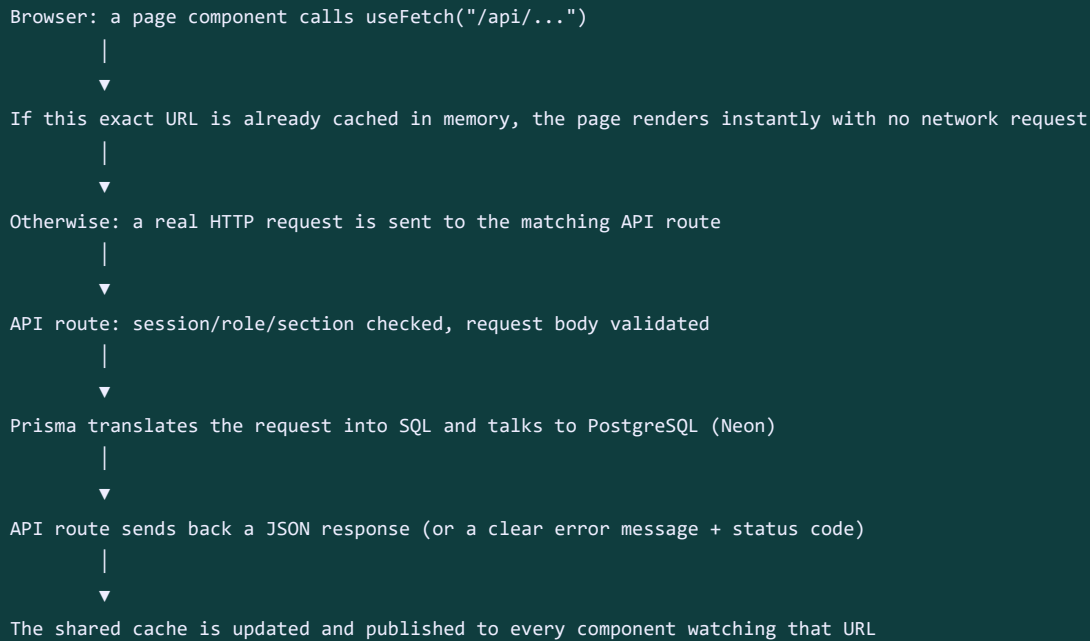
15.1 Per-entity query builders — one filter/sort definition per entity

Every paginated list route (invoices, products, brands, categories, vendors, purchase bills, purchase-bill payments, sales payments, credit notes, rate lists) is backed by a dedicated `src/lib/xQuery.ts` module exporting a `buildXWhere(filters) / buildXOrderBy(sort)` pair. Confirmed present as of this edition: `brandQuery.ts`, `categoryQuery.ts`, `creditNoteQuery.ts`, `paymentQuery.ts`, `purchaseBillQuery.ts`, `purchasePaymentQuery.ts`, `vendorQuery.ts`, and `rateListQuery.ts` (`buildRateListWhere / buildRateListOrderBy`) — plus `getInvoices() / getProducts()` in `db.ts` for the two original entities that predate this pattern. `src/lib/listQuery.ts` supplies the two cross-cutting helpers every one of these list routes shares: `parsePageParams(searchParams, maxPageSize?)` (clamps `page / pageSize`) and `monthYearToDateRange(month, year)` (turns a `?month= / ?year=` pair into a date-range filter).

The reason this exists as a named pattern, not just a convenience: **every list route ships with a companion /stats route**, and that stats route imports the *same* `buildXWhere()` the list route uses. A single page of paginated rows can no longer produce a correct running total client-side, so the stats route re-aggregates over every matching row server-side instead — and because both routes call into the identical builder function, the filter semantics used to decide "which rows count" literally cannot drift between the list and its own summary. Hand-rolling the `where / orderBy` logic separately in each route — even if it started out equivalent — would eventually diverge (a filter added to one and forgotten in the other) and produce a page whose stat cards silently disagree with what the list itself shows. This is treated as a hard rule (see CLAUDE.md's "Do Not" list): a route-local reimplement of filter/sort logic for an entity that already has a query-builder module is not permitted.

File uploads (purchase-bill attachments, the business logo) go to Vercel Blob storage. Every upload is checked not just by its declared file type but by inspecting the actual first bytes of the file ("magic bytes"), and every stored file URL is checked against a strict allow-list before being trusted, deleted, or rendered as a link.

16. Data Flow



Every mutation additionally calls `revalidateTag(tag, { expire: 0 })` (kept for consistency and in case server-side caching is added later — nothing currently reads from it) and writes an Activity Log entry.

Because every page is client-rendered, there is no separate server-side data cache actually in effect today — freshness comes entirely from the client-side cache in `src/lib/useCache.ts` plus each page explicitly refreshing after a write. The cache tags in current use are `"invoices"`, `"customers"`, `"products"`, `"vendors"`, `"purchase-bills"`, and `"reports"` — reports are busted on invoice, product, and purchase-bill mutations too, since the reports aggregate that data.

A `/stats` route has its own client-side cache key, separate from its list route's: `useFetch` caches by the full URL string, and a stats endpoint's query string differs from its list route's own. Consequently a list page that mutates data must call **both** `mutate()` (the list) and its stats hook's own `mutate()` (e.g. `mutateStats()`) after a write — `sales/invoices/page.tsx`'s `Promise.all([mutate(), mutateStats()])` is the reference pattern. Skipping the second call leaves the stat cards showing stale numbers even though the list below them has refreshed correctly.

16.1 `bustCache` vs. `bustCachePrefix` — a real, fixed bug

Update (2026-08-17): `bustCache(url)` (`src/lib/useCache.ts`) only ever invalidates the *one exact URL string* it's given — internally it's a single `cache.delete(url)`. This is the wrong tool the moment an endpoint's cached key carries query parameters beyond a fixed base, which by now describes almost every paginated or filterable list route (`/api/reports?type=outstanding&page=1&pageSize=20`, and equivalently every other `?search= / ?sort= / ?page=` list). Calling `bustCache("/api/reports?type=outstanding")` from a page that isn't the one holding that exact cached string is a silent no-op: the actual cached key in the shared `Map` is whatever full query string the reports page itself last requested, which will not match.

The fix is `bustCachePrefix(prefix)` , which walks every key currently in the cache and deletes it if it equals `prefix` exactly *or* starts with ``${prefix}?`` . This is the correct call from any page that doesn't own the cached URL outright — e.g. an invoice-create page busting the Reports page's cache before navigating away — and the caller must pass the bare endpoint path (`"/api/reports"`), not a partial query string, or the same silent-miss bug recurs.

This was an actual, live bug in the codebase, not a hypothetical: `sales/invoices/new/page.tsx` had been calling `bustCache("/api/reports?type=outstanding")` , which was a permanent no-op against the paginated outstanding-reports cache key — the invoice-create flow believed it was invalidating the Reports page's cache and never actually was. Fixed 2026-08-17 by switching that call to `bustCachePrefix("/api/reports")` . Any new code that needs to invalidate another page's paginated/filterable list cache from outside that page should use `bustCachePrefix` , not `bustCache` , from the outset.

17. PDF Generation

Invoice, credit-note, purchase-bill, and (as of 2026-08-11) rate-list PDFs are all built entirely inside the browser — nothing is rendered on a server. The technique, in plain terms:

```
The detail page already renders the document exactly as it should look, as ordinary HTML on screen
```

```
↓
```

```
html2canvas "photographs" that exact on-screen layout into a high-resolution image
```

```
↓
```

```
jsPDF pastes that image into a proper A4-sized PDF page, splitting content across multiple pages if the document is too long
```

```
↓
```

```
The result is returned as a downloadable file (a "blob")
```

Because printing, downloading, and emailing all funnel through this exact same image-capture pipeline (`generateInvoicePdfBlob()` in `src/lib/generateInvoicePdf.ts`), the document a customer receives by email is guaranteed to look byte-for-byte identical to what was printed or downloaded — there is no separate, possibly-inconsistent "print stylesheet" being maintained on the side.

17.1 Original vs. Duplicate copies

Indian invoicing convention often calls for multiple labelled copies of the same invoice — an "Original" for the buyer and a "Duplicate" retained by the seller. `PdfCopyDialog` lets the user choose either or both; if a Duplicate copy is included, a signature block for the receiver is automatically shown only on that copy. When an invoice is emailed to a customer, only the Original copy is ever attached.

17.2 Handling long documents

The page-splitting logic never cuts a table row in half across a page break, the footer/terms/bank-details block is always pinned to the very bottom of whichever page it lands on, and once an invoice has 18 or more line items, the system forces everything from the 18th item onward onto a fresh page even if a little more would technically still fit.

17.3 Rate List PDFs reuse the same pipeline

Rate Lists do not have their own PDF renderer. `RateListPrintArea.tsx` is a shared print-markup component (used both by the detail page, always mounted, and the list page's per-row Preview button, mounted on-demand: fetch the item list, mount the component, wait a frame plus `document.fonts.ready`, generate the blob, unmount) that gets pointed at by the exact same `generateInvoicePdfBlob()` function the invoice/purchase-bill pages use, targeting a `#rate-list-print-area` element instead of the invoice's own container. This works because `generateInvoicePdfBlob()` only *requires* a `<table>` with `<tbody>` rows to compute page-split boundaries — the invoice-specific bells (repeating `thead` / `tfoot`, `data-invoice-item-row`, `#invoice-page-marker`, multi-copy stamping) are all optional and no-op gracefully when absent, so no separate rate-list-specific PDF code path had to be

written. The rate list PDF header shows the business logo instead of a printed date, and — unlike the on-screen detail page, which does show a total for staff — the PDF itself has no total row, kept off intentionally.

17.4 PDF caching: `pdfCache.ts`, and why it must key on more than just the entity's own `updatedAt`

`src/lib/pdfCache.ts` is a client-side cache for generated PDF blobs, backed by IndexedDB so it survives page reloads within the same login session (cleared on sign-out — never persists across logins). PDF generation (`html2canvas` + `jsPDF`) is the expensive step, not the underlying data fetch (already cached by `useCache.ts`), so this module caches the rendered `Blob` itself, keyed by `${entity}:${entityId}` (`entity` is one of `"invoice"` | `"purchase-bill"` | `"return"` | `"rate-list"`) plus a **variant key** built by `buildPdfVariantKey(copyLabels?, extra?)` — a stable string derived from whatever render options and settings actually change the document's rendered content (copy labels, payment/return-history toggles, and every "still image data" input, sorted for a deterministic key).

Update (2026-08-09, commit `ca2b44d`): the invoice, credit-note, and purchase-bill PDF caches originally keyed their variant only on `settings?.updatedAt` (the business logo/settings) — sufficient for catching a settings change, but not for catching an edit to the *customer* or *vendor* record the document was issued to/from. A customer's address or GSTIN could be corrected after an invoice was generated, and the cached PDF (still showing the stale address) would keep being served indefinitely, since nothing about the invoice's own `updatedAt` or the settings' `updatedAt` had changed. Fixed by adding the linked customer's (or vendor's) `updatedAt` into the same variant key — e.g. `buildPdfVariantKey(undefined, { logo, settings: settings?.updatedAt, customer: invoice?.customer?.updatedAt })` — so editing the customer/vendor record produces a different key and forces a fresh render, the same way an edited invoice or a changed setting already did. This shipped alongside a batch of Sales/Purchases wording and form-order standardization in the same commit.

Rate List PDF caching (entity `"rate-list"`) follows the same shape: its variant key is `buildPdfVariantKey(undefined, { logo, settings: settings?.updatedAt, updatedAt: rateList.updatedAt })` — since a rate list's items are edited in place (bulk delete-and-recreate on save) rather than being immutable like a credit note's line items, its own `updatedAt` has to be part of the key too, or editing a rate list and returning to Preview/Download would keep serving the pre-edit PDF. The list page's own per-row Preview button deliberately does not use this cache at all — it always renders fresh, since it's a one-off, on-demand render rather than something a user is expected to click repeatedly on the same row.

`invalidateCachedPdf(entity, entityId)` drops every cached variant for one document (called after any edit/delete that could affect its rendered output), and `clearAllCachedPdfs()` wipes the entire cache on sign-out. A cache write/read failure is swallowed silently in every path — the cache is a pure optimization and must never be allowed to break PDF generation itself.

18. Email System

```
User clicks "Email Invoice" on the invoice detail page (or "Share PDF → Email" on a rate list)
|
▼
Browser generates the relevant PDF and attaches it, along with a recipient email, to a request
|
▼
Server checks: is this account allowed to send (not a manager), and have they sent too many emails recently?
|
▼
Server looks up Gmail credentials – first from Business Settings, falling back to environment variables if not configured
|
▼
If no credentials are configured anywhere, the request is rejected with a clear "Email not configured" message rather than a 503
|
▼
Nodemailer sends the email via Gmail's SMTP service, with the PDF as an attachment, using an escaped HTML email body
```

Every piece of text pulled into the email's HTML (customer name, invoice number, business name/address) is escaped via `src/lib/html.ts`'s `escapeHtml()` before being inserted, so no customer or business data can accidentally break the email's formatting or inject unwanted HTML/scripts into the message.

Credential source: the business's own Gmail address and app password, entered once in Settings and encrypted before being stored, are used first; two environment variables (`EMAIL_USER` / `EMAIL_APP_PASSWORD`) exist purely as a fallback for environments where Settings hasn't been configured yet.

18.1 `/api/send-invoice`

Sends the invoice's Original-copy PDF to the customer's own email; rate-limited at 20 sends per 15 minutes per user; gated by `requireWriteAccess()` (a `manager` cannot send email, matching the read-only role's general restriction).

18.2 `/api/send-rate-list` — a rate list has no linked recipient

A Rate List has no `Customer` / `Vendor` relation to default a recipient email from (Rate Lists are deliberately unlinked from the `Product` / `Customer` model — see Chapter 14.6), so `/api/send-rate-list` mirrors `/api/send-invoice` in every respect except where the recipient address comes from: the Share dropdown on the Rate List detail page opens a small `Modal` prompting for an address to be typed in at share-time, rather than autofilling one. Reading the route handler directly confirms the same shape as the invoice sender:

- `requireWriteAccess()` gate, then a per-user rate limit of 20 sends per 15 minutes (key `send-rate-list:${session.user.id}`) — the identical limit and window `/api/send-invoice` uses.
- Gmail credentials resolved the same way — `BusinessSettings.gmailUser` / `gmailAppPassword` first, then `EMAIL_USER` / `EMAIL_APP_PASSWORD` env vars, and a `503` with an explicit "Email not configured" message if neither is set.

- Server-side validation the client can't be trusted to have already done: a `to` address is checked against a basic email-shape regex, the uploaded PDF is capped at 10MB, and all three of `pdf / to / title` are required before anything is sent.
- The email body is built the same escaped-HTML way as the invoice sender (`escapeHtml()` around the title and the business's own footer text), attaching the PDF under a filename derived from the rate list's title with non-alphanumeric characters stripped.

19. Security

19.1 Authentication security

- Passwords hashed with bcrypt (cost 12) — never stored in readable form.
- A timing-safe "dummy hash" comparison (`DUMMY_HASH` , a fixed bcrypt hash compared against whenever the looked-up user doesn't exist) prevents attackers from telling apart "wrong password" from "no such account" by measuring response speed.
- Dual rate-limiting on login: per-account (8 attempts/15 min, keyed `login:${email}`) and per-IP-address (30 attempts/15 min, keyed `login:ip:${ip}` , with the IP itself resolved from `x-vercel-forwarded-for` / `x-real-ip` / `x-forwarded-for` in that order).
- Sessions are httpOnly, Secure (production), SameSite=Lax cookies containing a signed JWT, valid for 8 hours.
- Changing or resetting a password bumps a per-user `tokenVersion` counter, invalidating any already-issued session tokens for that account. Because re-checking `tokenVersion` against the database on every single authenticated request would add a full extra Postgres round-trip per request, the JWT callback only re-verifies it every 5 minutes (or immediately on an explicit client-triggered `update()`) — this bounds how long a reset/changed password takes to actually lock out an already-issued token to a few minutes, in exchange for not hitting the DB on every request.

Update (2026-08-12, commit 9bb66d7): `authorize()` in `src/lib/auth.ts` previously called `prisma.user.findUnique(...)` directly, with no handling for the lookup itself throwing. A transient database problem during a login attempt — a Neon cold-start, pooled-connection contention under the app's `connection_limit=1` setting, or a brief network blip — would surface as an unhandled exception from `authorize()` . Depending on how NextAuth's own error normalization behaved for a thrown-vs-returned-null failure, that risked either an ugly server error reaching the user, or (the more serious risk being guarded against) a distinguishable failure mode that could let an attacker tell "the database happened to be down" apart from "wrong password" by the shape of the response — the same category of information leak the dummy-hash comparison exists to prevent for account-enumeration. The lookup is now wrapped in its own `try/catch`: a DB error is logged server-side (`console.error("[auth] DB lookup failed during login attempt", ...)`) for diagnosis, and `authorize()` falls through to `return null` exactly as it would for a wrong password — `user` stays `undefined`, the subsequent `bcrypt.compare(...)` still runs against `DUMMY_HASH` so the timing profile is preserved, and the caller sees the same generic "Incorrect email or password" outcome NextAuth already produces for any other `null` return. A database outage during login can no longer be distinguished, by response shape or timing, from an ordinary failed login.

19.2 Authorization

All meaningful checks happen server-side via the four guard functions in `apiAuth.ts` (Chapter 15). A prior, documented security audit (dated 2026-07-03) found that roughly twenty API routes historically checked a session only to write to the activity log, but never actually blocked an unauthorized caller from proceeding; those fixes remain in place, including the login timing fix, the password-reset enumeration fix, and closing a privilege-escalation path where a client-supplied session update could previously have let a user grant themselves admin rights — the `jwt()` callback (see `src/lib/auth.ts`, lines documented above) always re-derives `name` / `email` / `role` from the database on a client-triggered `update()`, never from what the client sends.

19.3 Input validation / injection protection

- All database access goes through Prisma, which parameterizes every query — there is no hand-built SQL string concatenation anywhere in the reviewed code.
- Every form field has a defined, reused validation rule (Chapter 14.3), enforced identically on both browser and server.
- Email HTML bodies escape all user-supplied text before insertion (Chapter 18), for both `/api/send-invoice` and `/api/send-rate-list`.
- File uploads are checked against their real byte signature, not just their claimed file type, and every stored file URL is checked against a strict allow-list before being trusted or deleted.

19.4 Secrets at rest

Two especially sensitive fields — the Gmail app password and the bank account number — are encrypted (AES-256-GCM) before being written to the database, via `src/lib/crypto.ts`. An optional, dedicated `ENCRYPTION_KEY` environment variable, when set, derives its own key independent of the login-signing secret (`NEXTAUTH_SECRET`), so the two can be rotated separately. New values are written with a distinct `encv2:` prefix when this key is in use; older values (prefixed `enc:`, derived from `NEXTAUTH_SECRET`) continue to decrypt correctly either way, and legacy unprefixed plaintext values still pass through untouched — introducing `ENCRYPTION_KEY` requires no migration step. If `ENCRYPTION_KEY` is left unset, behavior is unchanged — the key is still derived from `NEXTAUTH_SECRET`. This remains accurate as of this edition; no further change to `crypto.ts` has landed since the 2nd Edition.

19.5 First-time setup safety

The one-time `/api/setup` endpoint (which creates the very first admin account) is completely disabled once the application is running in a production environment, and separately refuses to run at all if the database already has even one user in it.

19.6 Rate limiting — an honest limitation, confirmed still accurate

Rate limiting (on login, password reset, email sending — now including `/api/send-rate-list` alongside `/api/send-invoice`) remains a simple in-memory fixed-window counter (`src/lib/rateLimit.ts`), not a shared, persistent store like Redis. On a single-server deployment this is fine; across multiple simultaneous server instances, each instance keeps its own separate counter, so the effective limit is higher than the stated number, and a restart or redeploy resets all counters to zero. A Redis/Upstash-backed version (via `@upstash/redis`, with a graceful in-memory fallback when no Redis credentials are configured) was built, evaluated, and deliberately reverted at the project owner's request on 2026-07-24 — this remains the current state as of this edition. If distributed rate limiting is wanted later, the same fallback-shaped approach behind the existing `rateLimit()` function signature is the intended way to reintroduce it, so callers wouldn't need to change.

19.7 Default-deny middleware

`middleware.ts` sits in front of every `/api/**` request and rejects anything without a valid session unless the path is explicitly allowlisted as public (`PUBLIC_API_PREFIXES / PUBLIC_API_EXACT` — currently `/api/auth/*`, `/api/setup`, and `/api/settings/branding`). Per-route protection (`requireSession()` / `requireAdmin()` / `requireWriteAccess()` / `requireSectionAccess()`) still does the more

specific role/section checks; this middleware is a safety net underneath it, so a newly-added route is protected automatically even if a developer forgets to call one of those guards directly.

20. Error Handling

Frontend

Two distinct patterns are used consistently: a failure to even load a page's data shows a small error banner with a retry option; a failure that happens while saving something (a validation problem, a conflict, a server error) shows a temporary "toast" notification instead, so the user never has to scroll up to discover what went wrong on a long form.

Backend

Every API route wraps its logic in error handling that distinguishes between different failure types and returns an appropriate, specific HTTP status code and message rather than one generic failure for everything:

Situation	Status
Not logged in	401 Unauthorized
Logged in, but the wrong role/permission for this action	403 Forbidden
The record being requested/edited doesn't exist	404 Not Found
The request itself is invalid (missing field, bad format, business-rule violation)	400 Bad Request
Someone else updated this record since it was loaded on screen (optimistic-concurrency conflict)	409 Conflict
An email attachment is too large	413 Payload Too Large
Too many attempts in a short time (login, password reset, sending email)	429 Too Many Requests
A required external service (e.g. Gmail) isn't configured	503 Service Unavailable
Anything unexpected	500 Internal Server Error, with details only in the server log, never shown to the user

Optimistic-concurrency conflicts, explained

Several edit screens (invoices, purchase bills, customers, vendors, products, settings) send back the record's last-known "updated at" timestamp along with any edit. If someone else has changed that same record in the meantime, the save is rejected with a clear "this was changed by someone else" message rather than silently overwriting their change — protecting against two people editing the same record at the same time without realizing it.

Update (carried forward from the 2nd Edition, 2026-07-24, confirmed still true): editing an existing payment (`PUT /api/invoices/[id]/payment/[paymentId]`) previously ran as a sequence of separate database calls with no transaction wrapping them — unlike every other money-mutating flow in this codebase. It now uses the same `Serializable` transaction plus automatic retry-on-conflict pattern as recording a brand-new payment, so two concurrent edits (or an edit racing a new payment) can no longer both pass a stale balance check and together overpay an invoice. This fix has not regressed since — it remains in place in the current source

(`src/app/api/invoices/[id]/payment/[paymentId]/route.ts`) and is now the pattern every other money-mutating route follows, including invoice/purchase-bill number generation itself (Serializable transaction + P2034 retry).

Database

Prisma's own error codes are checked and translated into friendly messages where it matters — for example, trying to reuse a product SKU that's already taken returns a clear "SKU already in use" message rather than a raw database error.

Activity logging never breaks the real action

Writing an audit-log entry (`src/lib/activity.ts`) is deliberately wrapped so that it can never itself cause the real business action (creating an invoice, deleting a customer, etc.) to fail — if logging fails for any reason, it fails silently in the background rather than blocking the user's actual work.

21. Deployment

Development

<code>npm install</code>	also runs "postinstall": <code>prisma generate && node scripts/install-git-hooks.js</code>
<code>npm run dev</code>	starts the local development server
<code>npx prisma migrate dev</code>	applies a schema change locally
<code>npx tsx prisma/seed.ts</code>	fills a fresh local database with realistic demo data
<code>npm run test</code>	Vitest – unit tests + API integration tests
<code>npm run test:e2e</code>	Playwright – full-browser end-to-end tests

Production

Code is pushed to the main branch
|
Vercel automatically builds the project – the build script now runs
"prisma migrate deploy" (applies committed migration files) then "next build"
|
The Neon PostgreSQL database (pooled connection) receives any pending migrations
|
Vercel deploys the new version; environment variables (secrets/URLs) are configured
directly in Vercel's project settings, never committed to code

Update: the build script no longer runs `prisma db push`. It now runs `prisma migrate deploy`, which applies the project's own committed migration files instead of diffing the live schema and improvising the change. This switch was required once the schema gained real Postgres `GENERATED ALWAYS AS (...) STORED` columns (`Invoice.balanceDue`, `PurchaseBill.balanceDue`, `Product.isLowStock`) — Prisma's `@default(dbgenerated(...))` can describe a stored-generated column but `db push`'s schema-diffing engine cannot safely reconcile one, and would repeatedly attempt to `ALTER COLUMN` a column Postgres refuses to let it touch. Every schema change from this point forward must ship as a committed migration (`npx prisma migrate dev --name ...`) — `db push` is no longer a safe way to get a change into production.

The `postinstall` script (`prisma generate && node scripts/install-git-hooks.js`) is required for Vercel's build to succeed — removing the `prisma generate` half is explicitly called out in the project's own conventions as something that must never be done. The added `install-git-hooks.js` step is a local-developer convenience (wiring up pre-commit hooks) and is inert/harmless on Vercel's build machine.

A new `prebuild` script (`node -e "require('fs').rmSync('.next/dev', { recursive: true, force: true })"`) now runs automatically before every `npm run build`.

Update: this closes a real build-breaking bug, not just documents a workaround. Running `npm run dev` and then `npm run build` against the same `.next` folder could fail type-checking with `Module '"./routes.js"' has no exported member 'AppRouteHandlerRoutes'` inside `.next/dev/types/validator.ts`. That file is Turbopack dev-mode's own route-type-checking scaffold, generated only while `next dev` runs, and Next.js auto-appends `.next/dev/types/**/*.ts` to `tsconfig.json`'s `include` every time `next dev` starts — a developer cannot simply

delete that `include` entry by hand, since Next rewrites the whole file verbatim on the next `dev` run. A stale copy left over from a previous local `dev` session would then get type-checked against a fresh `next build` 's incompatible output and fail the entire build. The `prebuild` script fixes this at the root by deleting the stale `.next/dev` directory before every build runs, so this failure mode can no longer recur — no developer or CI step needs to remember a manual `rm -rf .next` before building.

A note on Windows local development

On Windows, the generated Prisma database client is a locked file while the development server is running — so after any schema change, the development server must be stopped, `npx prisma generate` run, and only then the server restarted, or the generate step will fail.

22. Third-Party Services

Service	Used for	Notes
Vercel	Hosting the entire application	Auto-deploys from the <code>main</code> branch
Neon	Managed PostgreSQL database hosting	Pooled connection string (<code>?pgbouncer=true&connection_limit=1</code>) used in production for efficiency
Vercel Blob	File storage for purchase-bill attachments and the business logo	Requires <code>BLOB_READ_WRITE_TOKEN</code> ; without it, attachment/logo uploads fail gracefully but nothing else in the app is affected
Gmail SMTP (via Nodemailer)	Sending invoice PDF emails, rate list PDF emails, and password-reset emails	Uses a Gmail "App Password," not a full account password or OAuth; credentials come from Business Settings with an environment-variable fallback
Razorpay's public IFSC directory	Looking up a bank branch name/address from an IFSC code when entering bank details	Called only server-side (admin-only proxy), never directly from the browser; 5-second timeout

Update: Rate Lists (Chapter 9's feature #32, see CLAUDE.md) are a new, separate PDF-producing feature since the 2nd Edition, so it is worth stating explicitly what did and did not change on the storage side: a Rate List PDF is generated **entirely client-side**, the same `generateInvoicePdfBlob()` pipeline the invoice/purchase-bill PDFs already use, and is never written to Vercel Blob. When a rate list is emailed (`POST /api/send-rate-list`), the freshly-generated PDF blob is attached to the outgoing Gmail message directly and is never persisted to storage first. Vercel Blob's actual footprint in this application is unchanged from the 2nd Edition: purchase-bill attachments and the business logo, and nothing else.

23. Environment Variables

Variable	Required?	Purpose
<code>DATABASE_URL</code>	Yes	Neon PostgreSQL pooled connection string (? <code>pgbouncer=true&connection_limit=1</code> in production)
<code>NEXTAUTH_SECRET</code>	Yes	Minimum 32 random characters. Signs sessions; also derives the legacy <code>src/lib/crypto.ts</code> key when <code>ENCRYPTION_KEY</code> isn't set
<code>NEXTAUTH_URL</code>	Production only	Full deployed URL, e.g. <code>https://your-app.vercel.app</code> — used to build password-reset links correctly
<code>ENCRYPTION_KEY</code>	Optional	A dedicated key for encrypting secrets-at-rest (Gmail app password, bank account number) independently of <code>NEXTAUTH_SECRET</code> , so the two can be rotated separately. Without it, encryption still works exactly as before, keyed off <code>NEXTAUTH_SECRET</code> — no migration is needed either way
<code>EMAIL_USER</code> / <code>EMAIL_APP_PASSWORD</code>	Optional (needed for email sending)	Fallback Gmail address + App Password, used when Business Settings hasn't configured its own <code>gmailUser</code> / <code>gmailAppPassword</code>
<code>BLOB_READ_WRITE_TOKEN</code>	For purchase-bill attachments/logo	Authorizes uploads to Vercel Blob storage; auto-provided on Vercel, pulled locally with <code>vercel env pull</code> . Without it, attachment/logo upload fails but everything else works

This table is unchanged from the 2nd Edition apart from formatting — both `ENCRYPTION_KEY` and `BLOB_READ_WRITE_TOKEN` were already correctly documented as of 2026-07-24 and remain accurate against the current `CLAUDE.md` and source.

24. Project Dependencies

Package	Why it's in this project
<code>next (16.2.9)</code> , <code>react / react-dom (19.2.4)</code>	The core framework and UI library
<code>@prisma/client</code> , <code>prisma (^5.22.0)</code>	Database access and schema management
<code>next-auth (^4.24.14)</code> , <code>@auth/prisma-adapter</code>	Login/session handling
<code>bcryptjs</code>	Password hashing
<code>nodemailer (+ @types/nodemailer)</code>	Sending email (invoices, rate lists, password resets)
<code>jspdf</code> , <code>html2canvas</code>	Building invoice/credit-note/rate-list PDFs in the browser
<code>exceljs</code>	Generating downloadable Excel reports and rate-list Excel exports/imports
<code>jszip</code>	Bundling the GST filing package into a ZIP
<code>@vercel/blob (^2.5.0)</code>	File storage for purchase-bill attachments and the business logo
<code>dotenv</code>	Loading environment variables during local scripts (e.g. the seed script)
<code>@types/bcryptjs</code>	Type definitions for <code>bcryptjs</code>
<code>tailwindcss / @tailwindcss/postcss (dev only)</code>	Present in the toolchain, though the app's actual styling approach is CSS Modules + custom properties rather than Tailwind utility classes
<code>typescript</code> , <code>eslint</code> , <code>eslint-config-next (dev only)</code>	Type-checking and code-quality tooling
<code>vitest</code> , <code>@types/node</code> , <code>@types/react</code> , <code>@types/react-dom (dev only)</code>	Unit and API integration test runner and its type support
<code>@playwright/test</code> , <code>playwright (dev only)</code>	Full-browser end-to-end testing

Update: the dependency list itself has not meaningfully changed since the 2nd Edition — no new runtime package was added for Rate Lists, configurable document numbering, or credit notes; all of it was built on the same `jspdf / html2canvas / exceljs / jszip` toolkit already in the project. What *has* changed is the `scripts` block in `package.json`, verified directly against the current file:

- `"prebuild": "node -e \"require('fs').rmSync('.next/dev', { recursive: true, force: true })\"` — new, see Chapter 21.
- `"build": "prisma migrate deploy && next build"` — changed from `prisma db push && next build`, see Chapter 21.
- `"postinstall": "prisma generate && node scripts/install-git-hooks.js"` — gained the git-hooks installer step alongside the still-mandatory `prisma generate`.
- `"test"`, `"test:watch"`, `"test:coverage"`, `"test:e2e"`, `"test:e2e:ui"` — the Vitest/Playwright test scripts introduced with the testing infrastructure (2026-08-09), present but not yet exercised against a real database as of this edition (see Chapter 28).

`@vercel/blob` is confirmed still at `^2.5.0` and still used only for purchase-bill attachments and the business logo (see Chapter 22's Update) — Rate List PDFs do not add a new dependency or a new use of Blob storage.

25. Business Workflow — A Day in the Life

Morning: Owner checks the Dashboard – revenue this month, outstanding balances, overdue invoices, low-stock alerts, at a glance

|
A shipment of chemicals arrives from a vendor -> a Purchase Bill is created, stock automatically increases

|
A prospective customer asks "what do you have and at what price?" -> Staff builds a Rate List (free-text items – name/brand/unit/list rate/discount%, independent of the Product catalog), previews the PDF, and emails it directly from the app via Share -> Email – no invoice or customer record is created yet, since nothing has been sold

|
That prospect calls back ready to order -> Staff creates a new Invoice, GST calculated automatically (with the invoice number shaped however this business has configured it – see below), stock automatically decreases

|
The invoice is emailed directly to the customer, or printed for a walk-in customer

|
The customer pays a few days later (by cheque/UPI/bank transfer) -> the payment is recorded against the invoice

|
A different customer returns a damaged item -> a Credit Note is issued (its own auto-numbered document, e.g. CN-2026-27-0004), stock is restored

|
End of month: the owner pays the vendor's bill -> payment is recorded, the vendor's outstanding balance drops

|
Owner reviews Sales/Purchase Reports and, at tax time, generates the GST Filing package (Sales/Purchase Register, B2B/B2C split, Credit Notes, HSN Summary, ZIP download) to send to the accountant

|
Occasionally: something is deleted by mistake -- it sits safely in the Recycle Bin, fully recoverable (indefinitely for invoices/purchase bills/credit notes, since their numbers are legally significant GST sequences; 30 days for everything else)

Update: this narrative gained a new step since the 2nd Edition — the Rate List step above. A Rate List is deliberately *not* the same thing as an invoice: it exists so staff can hand a prospective or existing customer a price sheet/quotation without that interaction ever touching stock, GST, or the customer database. Items on a Rate List are free text (a supplier's own brand/unit spelling like "500 GM" doesn't need to match this app's Product / Brand records), can be typed by hand, pasted from an Excel table, or bulk-imported from an uploaded .xlsx / .csv , and the finished document can be downloaded, previewed, or emailed with no linked customer required — the Share -> Email flow simply prompts for an address at send time.

Also worth narrating explicitly: the invoice number the customer sees in the second step above is no longer a hardcoded shape. An admin can choose, per document type (invoice, purchase bill, credit note), between three layouts (SH-2026-27-0001 , 18/2026-27 , or SH-18/2026-27), set a custom prefix, and issue a one-time "start numbering from" override — all

from Settings -> Document Numbering. Whatever shape is chosen, the year segment always follows the Indian financial year (1 April – 31 March), so numbering for a given layout resets at the same boundary GST filing periods do, not on 1 January.

26. Non-Technical Explanation

For a shop owner

Think of Science Hub as a smart notebook that never makes an arithmetic mistake, never runs out of pages, and remembers who wrote what and when. Instead of writing an invoice by hand and doing GST maths on a calculator, you type in what was sold, and the system works out the tax, gives the bill its next number automatically (in whatever style you've told it to use, and it never forgets to reset that numbering when a new financial year begins), and quietly keeps count of how much stock is left on your shelf — all at the same moment.

You're not limited to selling from your existing product list, either. If a customer just wants a price list to compare against — a quote, not a sale yet — you can build a **Rate List**: a simple, sharable price sheet with whatever items, brands, and prices you type in, downloadable as a PDF or emailed straight from the app, without it ever touching your actual stock or GST records.

For a business owner thinking about risk

Nothing is ever truly deleted by accident — anything removed goes into a "Recycle Bin" before it's gone for good (indefinitely for invoices, purchase bills, and credit notes, since a gap in those numbers could raise questions at tax-filing time; 30 days for everything else), and every single action anyone takes is written down in an activity log with their name and the time. You can also decide exactly what each staff member is allowed to see and do — someone can be given permission to just look at reports without being able to change anything at all.

You also get to decide how your invoice, bill, and credit note numbers look — some businesses already have a numbering habit from a previous system, and you can match it instead of being forced into one fixed style, without ever losing track of which financial year a number belongs to.

For a school student, using an analogy

Imagine a library. The database is the shelves where every book (piece of information) is actually kept. The website you see in your browser is like the front desk — you never touch the shelves yourself. When you ask the front desk for a book (say, "show me this month's invoices"), a librarian (the API) goes and fetches exactly that from the shelves and brings it back to you, checking your library card (your login) first to make sure you're allowed to see it.

A Rate List is like a flyer the librarian can print for you on request — it lists books and prices from a catalog you made up on the spot, but it isn't the library's real shelf inventory, so handing one out doesn't change what's actually on the shelves.

For someone who has never written a line of code

Every button you click in this application quietly sends a small, structured message to a computer somewhere else (the "server"), which checks who you are, does some maths or looks something up in an organized filing system (the "database"), and sends back the answer — which is then drawn on your screen as the numbers and tables you see. Nothing you see is invented on the spot; it always comes from a real, permanent record kept in that filing system.

27. Developer Guide

27.1 First-time setup

1. `npm install` (also runs `postinstall: prisma generate` and installs the project's git hooks, automatically)
2. Create a `.env` file with at least `DATABASE_URL` and `NEXTAUTH_SECRET`
3. `npx prisma migrate dev` (creates the database tables)
4. `npx tsx prisma/seed.ts` (fills the database with realistic demo data, prints an admin login)
5. `npm run dev` (starts the app at `http://localhost:3000`)

27.2 First admin account in a fresh production deployment

Send a one-time request to `POST /api/setup` with a name/email/password. It only works while the database has zero users, and is completely disabled once the app is running in production with even one user already existing — so it can safely be left in the codebase without being a lingering security risk.

27.3 Making a schema change

1. Edit `prisma/schema.prisma`
2. Stop the local dev server (Windows locks the generated client file while it runs)
3. `npx prisma migrate dev --name describe-your-change`
4. `npx prisma generate`
5. Restart the dev server

This is unchanged from the 2nd Edition, and remains the *only* safe path to production — see Chapter 21's Update on why `prisma db push` can no longer be relied on once real Postgres generated columns exist in the schema.

27.4 Adding a new page

1. Create `src/app/(dashboard)/<section>/<page>/page.tsx`, starting with `"use client"`.
2. Fetch data with `useFetch("/api/<resource>")`; call `mutate()` after any write.
3. Add an entry to `NAV_GROUPS` in `src/components/layout/DashboardShell.tsx` if it should appear in the sidebar.

27.5 Adding a new API route

1. Create `src/app/api/<resource>/route.ts`.
2. Call the appropriate guard from `src/lib/apiAuth.ts` first, before touching any data — `requireSession()`, `requireAdmin()`, `requireWriteAccess()` (blocks the `manager` role), or `requireSectionAccess(section)`.
3. Validate the request body — reuse an existing `rules.*` validator from `src/lib/validation.ts` where possible.
4. Call `logActivity(...)` for any create/edit/delete.
5. Call `revalidateTag(tag, { expire: 0 })` after any write (always the two-argument form).

6. If the route doesn't call one of the guards in step 2, it must instead be added to `middleware.ts`'s public allowlist — otherwise the default-deny middleware will 401 it, which is the intended safety net, not a bug.

27.6 Adding a new paginated list endpoint

The established pattern for every list route added since the pagination refactor (invoices, products, brands, categories, vendors, purchase bills, purchase-bill payments, sales payments, credit notes, rate lists):

1. Add a `buildXWhere(filters)` / `buildXOrderBy(sort)` pair to a new or existing `src/lib/xQuery.ts` — never inline the filter/sort logic directly in the route handler, so the list route and its stats route can't drift apart.
2. In the list `route.ts`, parse query params with `parsePageParams(searchParams, maxPageSize?)` and `monthYearToDateRange(month, year)` from `src/lib/listQuery.ts`, call the builder, and return `{ data, total }` — never a bare array, since a single page of paginated rows can no longer produce a correct total client-side.
3. Add a companion `stats/route.ts` under the same folder that reuses the *same* `buildXWhere()` to aggregate over every matching row, independent of the list route's current page.
4. On the client, debounce the search input with `useDebounceValue()` from `src/lib/useDebounceValue.ts` before it hits the API.

27.7 Adding a Rate-List-adjacent bulk-entry feature

Rate Lists already support two ways to bulk-populate an items table without retyping a large supplier price list by hand: pasting tab-separated text copied from Excel, and uploading an `.xlsx` / `.csv` file. Both routes — the client-side paste parser and the server-side `POST /api/rate-lists/parse-import` handler (which uses ExcelJS to read the uploaded sheet) — funnel through one shared parser, `parseRateListRows()` / `parsePastedRateListText()` in `src/lib/rateListImport.ts`. It detects a header row by column-name matching (Name/Brand/Unit/Discount/List Rate, in any order or casing) and falls back to a positional guess by column count when no header is recognized.

Any future rate-list-adjacent bulk-entry feature — a third import format, a different paste shape, an import feature for some other free-text entity — should extend this one parser rather than writing a second, parallel one. Imported rows should always be merged into the form's existing in-memory state for the user to review and edit before saving, the same as a manually-typed row, rather than written to the database directly.

27.8 Rules that must never be broken (confirmed still true in the current code)

- Never assume the invoice/purchase-bill/credit-note number has a fixed shape (e.g. `SH-YYYY-0001`) — the layout itself is admin-configurable per document type (Settings -> Document Numbering). Always go through `computeNextNumber()` / `numberFormatDbFilter()` / `findMaxSequence()` in `src/lib/documentNumbering.ts` rather than hand-rolling string splitting, and never let a document's own date cross a financial-year boundary once its number is generated.
- Never remove the `postinstall` script (or its `prisma generate` half) from `package.json` — Vercel's build depends on it to generate the Prisma client.
- Never mutate a product's stock outside of `batchAdjustStock()` — the ledger must always stay the true, complete history of every stock change, using the most specific `StockMovementType` for the action.
- Never accept or delete a file URL for a purchase-bill attachment or the business logo without passing it through `isPurchaseBillBlobUrl()` / `isLogoBlobUrl()` + `deleteAttachmentBlob()` in `src/lib/blobStorage.ts` first.

- Always use the two-argument form of `revalidateTag(tag, { expire: 0 })` — the single-argument form is deprecated in this version of Next.js.
- Never rely on `prisma db push` to reach production — the schema now contains real Postgres `GENERATED ALWAYS AS (...) STORED` columns that `db push`'s diffing cannot safely reconcile; always ship a committed migration via `prisma migrate dev`.
- Never trust a client-supplied `isInterState` flag on invoice create/edit — always derive it server-side via `deriveIsInterState()` in `src/lib/gstLocation.ts`.

Update: `CLAUDE.md` and this Bible have both continued to track the application closely since the 2nd Edition's correction pass. As of this edition, `CLAUDE.md` additionally documents Rate Lists (with their deliberate non-relation to the `Product` catalog), the app-wide fullscreen `Modal` convention, the shared detail-page action-toolbar convention, the mobile breakpoint standardization (1024px sidebar/topbar threshold, 768px form-field compact-mode threshold), and the current, real Pending Tasks list reproduced in Chapter 28 below. New readers should still verify anything critical against the live source, but no stale references (AI bill-scan, Railway hosting, Factory Reset) have crept back in.

28. Future Improvements

This chapter previously tracked a list of open observations from the 2nd Edition (2026-07-24), nearly all of which were implemented the same day. This edition replaces that list with the project's actual, current Pending Tasks and Known Issues, taken directly from `CLAUDE.md` as of 2026-08-17 — not a fresh audit's speculative wishlist.

Carried over from the 2nd Edition and still genuinely open

- **Multi-user simultaneous notifications** (e.g. "someone else is editing this invoice right now") to make the existing optimistic-concurrency conflict detection feel proactive rather than something only discovered after clicking Save — this needs a real-time channel (polling or WebSockets) the app doesn't currently have, and remains explicitly deferred pending a design decision on scope.
- **Scheduled/automatic reminder emails for overdue invoices**, building on the email infrastructure that already exists.
- **AI & automation opportunities** (still open, unchanged): automatic GST-rate/HSN suggestions when adding a new product, and anomaly flags on the dashboard (e.g. "this invoice's total is unusually high for this customer") drawing on the same historical data already collected in the ledger and reports.

New since the 2nd Edition

- **Provision a real test database.** `.env.test.example` exists and both the Vitest (`tests/api/**`) and Playwright (`tests/e2e/**`) suites are fully written and pass type-checking/lint, but neither has ever been run against a live database — no test database was available at authoring time. Copying it to `.env.test` pointed at a disposable Neon branch and actually running `npm run test / npm run test:e2e` is the next concrete step, not further test-writing.
- **Expand test coverage.** The initial coverage pass unit-tested `src/lib/validation.ts`, `invoiceCalc.ts`, `roundOff.ts`, `gstLocation.ts`, `stockStatus.ts`, `purchaseBillForm.ts`, and `listQuery.ts`, and integration-tested the `customers / vendors / invoices / bin` routes. Purchase bills, payments, returns/credit notes, rate lists, and the bin restore/permanent-delete routes have no tests yet.
- **Rate Lists follow-up phases**, deliberately deferred from the 2026-08-11 MVP scope (email-send has since shipped via `/api/send-rate-list`, so it is no longer on this list):
 - A public, token-based share link so a customer can view/download a rate list without logging in (a `RateList.shareToken` -style field, rate-limited, and added to `middleware.ts` 's public allowlist).
 - Recycle Bin integration for rate lists — `DELETE /api/rate-lists/[id]` already soft-deletes via `deletedAt`, but there is currently no restore UI for it anywhere in the Bin page.
- **Wire up `FormField` /error-state UI for `RateListItemsTable` 's paste-import.** Its submit button is currently disabled purely on `!pasteText.trim()`, with `validate()` running but no field-level error surfaced on an empty submit — a silent no-op rather than the app's normal inline-error pattern. This needs fixing before the button can drop the `.trim()` disable guard and follow the same "new/create form" convention (disable only on `saving`, let validation surface errors) already applied to every other quick-add modal in the app, including the Categories/Brands Add & Rename modals, which had this exact gap fixed on 2026-08-14.

Resolved since the 2nd Edition — no longer applicable

- ~~Theme flicker on initial load~~ — already fixed prior to this edition: `src/app/layout.tsx` sets `.dark / --c-accent` via a pre-hydration inline script plus `suppressHydrationWarning` before first paint. Retained here struck through, matching `CLAUDE.md`'s own phrasing, rather than silently dropped, in case of a future regression to watch for.
- ~~`npm run build` can fail type-checking against a stale `.next/dev` directory left over from a previous `npm run dev` session~~ — this was an open, unresolved problem as of the 2nd Edition's Chapter 21 draft; it is now fixed at the root by the `prebuild` script (see Chapter 21), not merely documented as a workaround. No manual `.next` cleanup step is needed before building.
- The 1st Edition's item about Factory Reset and manager accounts remains moot — the Factory Reset feature stays removed from the application entirely and does not reappear anywhere in the current source or this document.

This is the final chapter of the Science Hub Bible, 3rd Edition.